Fuzix for the Pico Computer

Unix and BBC BASIC on the Pico Computer 2 and 3

Release v0.11 — August 2026

Contents

1 Introduction

Fuzix is Alan Cox's compact V7-style Unix. This port makes it a third first-class environment for the Pico Computer, beside MMBasic and MicroPython: a genuine multi-tasking Unix booting from SD card, with a full set of classic utilities — and R. T. Russell's BBC BASIC as its flagship application, complete with BBC graphics modes on the HDMI display, the four-channel BBC sound system on the audio DAC, joystick and analogue inputs, and a spare serial port.

It also compiles on its own. `cc` is a complete C89 toolchain running on the hardware — not a cross-compiler — and it generates native ARM code, so the machine builds its own programs, at its own speed, with no PC involved. `mmbc` puts MMBasic in front of it: an MMBasic program translates to C, compiles, and runs as a native program several times faster than the interpreter it was written for.

One kernel image serves both machines. At boot it probes GP27 for the DS3231's 32 kHz clock (the same test MMBasic and MicroPython use): present means Pico Computer 3, absent means Pico Computer 2. The practical difference is the SD card wiring, handled automatically; the boot banner names the detected board.

Headline specification as configured here:

- CPU at 375 MHz (the XGA-capable clock, as MMBasic uses)
- 340 KB of program RAM managed as 4 KB pages, with the 8 MB PSRAM behind it — up to 64 concurrent processes, and a single process may have about 292 KB of it. A swapped-out process is a PSRAM allocation the size of the process, not a slot on a device, so nothing is reserved for one that does not exist
- Programs get their heap from the PSRAM too, so a BASIC array or a C `malloc` is limited by the 8 MB rather than by the process
- Root filesystem on SD card; the on-board NAND flash holds a recovery system
- 80×40 colour ANSI console on HDMI, mirrored to the USB-C serial port, with USB keyboard support (six layouts)
- Pre-emptive multitasking: a runaway program can always be stopped from the keyboard
- A self-hosted C89 compiler generating native ARM code, and an MMBasic translator in front of it — both run on the machine itself
- MMBasic's own full-screen editor, `mmedit`, so BASIC is written, translated, compiled and run without leaving the machine

1.1 Since v0.11 — not yet released

Written down here rather than folded into the list below, because the published v0.11 card does not have it:

- **I2C2** works from BASIC, with MMBasic's option word and its OPEN timeout — see the I2C2 section for the two deliberate differences from a PicoMite. A BMP180 on GP38/GP39 reads through MMBasic's own reference program unchanged apart from the bus.
- **The RTC alarm** is armable and was re-tested properly. The v0.11 note that it worked was taken from a test that could not tell an alarm from the DS3231's 1 Hz square wave, and had in fact been measuring the square wave.
- **`printf` understands `%lld` and `%llX`.** It did not, silently, which made every `&H` constant zero in a program translated *on the board*.

- **SPI** on the header pins — see the SPI section. An ILI9341 driven from BASIC fills its 240×320 screen in 26 ms.
- **Every peripheral got faster.** `clk_peri` had been left on the USB PLL at 48 MHz whatever the system clock was doing, so SPI could not exceed 24 MHz, and the UART and SD card were on the same 48 MHz. It now follows `clk_sys`.
- **The built-in fonts can be read.** `MM.INFO(FONT ADDRESS n)` gives the address of a font's glyph data, and `PEEK` — which did not exist at all before — reads it. Together they let a program draw MMBasic's own text onto a display the firmware knows nothing about. Both spellings are MMBasic's. See the section on the glyphs, and the worked barometric station that uses them.

1.2 New in v0.11

This release is about the **outside world**. The machine could already draw, make a noise and compile its own BASIC; what it could not do was control anything. It can now — and the pin work is between one and two orders of magnitude faster than the way it used to be done, because it stopped being a system call.

- **Pins are owned, and come back.** `SETPIN` claims a pin from the kernel, and the claim is released — with the pin **reset to an input** — when your program ends, however it ends, including being killed or crashing. A program that dies driving a relay no longer leaves it driven. Claiming a pin another program holds is refused rather than quietly fought over, and only the I/O header can be claimed at all, so a BASIC program can no longer take a pin the SD card or the display is using.
- **A pin is now a register access, not a system call** — about ten nanoseconds against 1.5 µs. That is what makes bit-banging a protocol from BASIC possible at all, and it is why the rest of this list could be built without adding kernel drivers for any of it.
- **Analogue in**: `SETPIN n, AIN` reads volts, `ARAW` the raw count, using MMBasic's own ten-sample sort-and-discard filter.
- **Pin interrupts**: `SETPIN n, INTH|INTL|INTB, handler`, with the handler an ordinary `SUB`.
- `SETTICK` — four periodic timers, with `PAUSE`, `RESUME` and catch-up that drops missed ticks rather than queueing them. Measured at 1999.98 ms for twenty 100 ms ticks.
- `ON KEY`, both forms — one that fires on any key and leaves it for `INKEY$`, one that claims a single key and eats it.
- **`INKEY$` works.** It never had, on this machine: the console's line editor was taking every keystroke before a program could see it, and nothing had exercised it hard enough to notice. A program reading keys now holds the terminal, and gives it back for `INPUT` and at exit.
- `PWM` and `SERVO`, with MMBasic's arithmetic and its frequency range — 50 Hz to 100 kHz verified on a scope, and servo positions from 0.8 ms to 2.2 ms.
- **`PULLUP` and `PULLDOWN`** on the input modes, and hysteresis on every input, as MMBasic does.
- **GP32**, the DS3231's alarm line, is readable. (Setting an alarm is not yet possible — see the pin section.)
- `/dev/i2c` for C programs, sharing the QWIIC bus with the kernel's own clock.

Two things this exposed and fixed along the way: the compiler could be run twice at once and corrupt its own output, and its progress dots ignored redirection. Both are gone.

1.3 New in v0.10

This release fills in the MMBasic translator's two largest gaps — the rest of the drawing statements, and structures — and makes the whole machine faster.

- **`BOX`, `RBOX`, `TRIANGLE` and `ARC` translate.** With `CIRCLE`, `TEXT` and `MAP` already in, the PicoMite drawing set is now covered. Each primitive is a header of static C functions, one header per primitive, so a compiled program carries only the primitives it actually uses — the compiler discards the rest. The same headers are callable from plain C; the C manual documents them.
- **`TYPE ... END TYPE` — MMBasic structures.** Nested types, member arrays, structure arrays, `LOCAL`s, by-reference parameters, whole-structure assignment, `STRUCT COPY/CLEAR/SWAP`, and `STRUCT(SIZEOF/OFFSET/TYPE)` folded to compile-time constants. The firmware's byte layout is reproduced exactly — sizes, offsets, padding and the length-byte string format — so `STRUCT(SIZEOF "t")` answers the same number here and on a PicoMite, and assigning an over-length string to a `LENGTH n` member raises `String too long` just as the firmware does. Verified side by side against a real PicoMite running the firmware's own structure test suite: every feature implemented here passes identically there. What does not translate is refused by name rather than mistranslated; the translator chapter lists it.
- **A class of board-only crashes in compiled programs is fixed.** `bcrun` could size a program's memory arena so that the stack ended up misaligned for the ARM's paired-register stores, and the Cortex-M33 faults on those regardless of its unaligned-access support. Neither the host runner nor qemu enforces that, so the crash existed only on real hardware. The arena is now rounded so the stack is always 8-byte aligned.
- **`ON ERROR SKIP/IGNORE`, with `MM.ERRNO` and `MM.ERRMSG$`.** A program can survive an error instead of stopping at it, exactly as it can on a PicoMite — and the errors themselves now fire where the interpreter fires them. That was the larger half of the work: float division by zero used to answer `inf` rather than stopping, `SQR(-1)` and `LOG(0)` answered `nan` and `-inf`, string concatenation past 255 characters truncated instead of erroring, and `ASC("")` errored where a PicoMite returns 0. Each of those was a program that behaved differently here and said nothing about it. There is a section on all of this below.
- **The arithmetic checks are paid for only by programs that can use them.** A program with no `ON ERROR` cannot trap an error, so `mmbc` compiles its float divisions to the machine's own divide and its `SQR`/`LOG`/`ASIN`/`ACOS` to direct libm calls — full speed, C answers (`inf`/`nan`) if the argument was bad. The moment `ON ERROR` appears, every one of those becomes the checked, trappable statement the interpreter has. "What `ON ERROR` costs", below, has the numbers.
- **The machine can say what it is.** `MM.VER` gives the release as a number (0.10 reads higher than 0.09), `MM.DEVICE$` gives `Fuzix on PC2` or `Fuzix on PC3` — detected, not assumed — and `MM.CMDLINE$` gives the arguments the program was started with, so a translated program can be used as a proper command. `MM.DEVICE$` needed a new kernel call: which board this is was something the kernel knew and only the boot banner ever said.
- **`mmedit`'s Find works from the machine's own keyboard.** `F3` opened the prompt and then ignored Enter, because a serial terminal sends CR where the USB keyboard sends LF and the prompt only accepted the first. Every prompt now takes either, so `F3`, `F10` and the rest behave the same on the HDMI console as over a serial line.
- **`mmedit` takes the screen back from a graphics mode.** Editing after a program that ended in `MODE 2` meant typing into a console the monitor was not showing. The editor now switches to `MODE 1` while it runs and restores the mode when it leaves, so the program's screen comes back as it was.

- **`mmedit` colours the new statements correctly.** `Box`, `RBox`, `Triangle`, `Arc`, `Type`, `End Type`, `Struct` and `Struct(` now show as translatable (cyan) rather than interpreter-only (blue), and so do the `MM.` functions `mmbc` understands — while `MM.WATCHDOG`, `MM.INFO` and the rest it does not are honestly blue, where the editor used to call everything translatable.
- **The kernel's crash report leads with `r4`–`r7`** — the registers that actually locate the fault in compiled code — so a report copied from the screen is useful even when cut short.

And a group of changes that make everything on the machine faster, without any program being written differently:

- **The C library's string routines are newlib's.** Every program in the userland — the compiler, the translator, `bcrun`, the editor, your own C — was using the generic byte-at-a-time `memcpy`, `memset`, `memmove`, `strlen`, `strcmp` and their relatives: about nine cycles a byte on this processor. Twenty-one objects are now taken from the ARM toolchain's own `libc.a` at build time, so the machine runs the routines that were written and tuned for it. On identical bytecode: the grains benchmark 52,052 → **55,163**, the solar eclipse predictor 2.022 → **1.953 s**, a graphics frame 78.81 → **74.03 ms**. The compiler is a program in that userland too, so compiling the eclipse went from 7 seconds to 6.
- **Integer division uses the divide instruction.** MMBasic integers are 64 bits wide and this processor's `SDIV` is 32, so every `\` and `MOD` went through a software routine that is roughly twenty times the cost of the instruction — even when both numbers were small. When they both fit in 32 bits, which in practice is nearly always, the instruction is used directly. The grains benchmark gains about 1%, the eclipse predictor 2%.
- **`PIXEL` no longer pays a system call per point.** Plotting a point cost about 1.3 µs of crossing into the kernel to store 15 ns of pixel — 96% overhead. Points now queue and go across in one call, up to 512 at a time or every 10 ms, whichever comes first, and any other drawing statement flushes the queue first so the order on screen is exactly the order in the program. A pure-`PIXEL` demo (`ripple.bas`) went from 217 ms a frame at v0.9 to **171** — 21%. A program that already builds its points into an array and plots them with the array form of `PIXEL` is unaffected: that was, and remains, the fastest way to plot a lot of points. One deliberate behaviour change comes with this: a `PIXEL` outside the screen is now **dropped**, as the interpreter drops it, where before the coordinates wrapped and the point appeared somewhere else. A program that relied on the wrapping will look different.
- **Recursion goes 255 levels deep, not 15.** The old limit was nothing to do with the stack. A by-reference argument occupies a slot that cannot be released until the call returns, there were sixteen slots, and so a routine that passed an argument by reference to itself stopped at fifteen. There are now 256, which costs 2 KB and is measured at 255 levels for every shape of recursive routine — with locals, with local strings, with a string expression built at each level. Beyond that a program stops with `Expression too complex` rather than misbehaving. For scale: a recursive walk over a million-node balanced tree is twenty levels deep. A program that recurses without any terminating condition is still a program that crashes the machine; the depth guard does not cover compiled routines that the translator has turned into native code.
- **Four C89 gaps in the preprocessor.** `?:` in an `#if`, the `#line` directive, `__LINE__` inside an `#if`, and macro arguments being expanded before `##` pastes them. The host build of the compiler uses `gcc -E`, so the machine's own preprocessor had never been tested by anything; it has its own test suite now. C conformance measured on the machine goes from 156 to **160 of 175**.
- **`cc1` folded signed constant division as unsigned.** `-7/2` in a constant expression came out as an enormous positive number. Both operands are now cast to a signed type before folding.

1.4 New in v0.9

This release takes the 32 MB limit off the filesystem and makes compiled code substantially faster.

- **A 2 TB disk format.** Block numbers are 32-bit and inodes 256 bytes, in place of the 16-bit block number that capped a filesystem at 32 MB. The shipped card now has an 800 MB root rather than 32 MB, a single file can reach 1 GB, and the format itself goes to 2 TB. **Old cards and new kernels do not mix**: each refuses the other by name rather than misreading it, so flash the kernel and write the card in the same sitting.
- **Compiled code is over 60% faster.** Dhrystone 2.1, compiled on the machine, went from 102,842 to 166,834 per second — 44% of the same benchmark cross-compiled by `gcc -O2` for this chip. Almost none of that came from generating cleverer instructions; it came from *keeping hot code native*. Library symbols are bound once at load instead of being looked up on every call, the hottest local in a function is held in a register, and compound assignment on 64-bit and double values is inlined rather than calling out to the runtime — that last one had been costing every MMBasic `FOR` loop a crossing out of native code on every single iteration.
- **The console survives a lost UART interrupt.** Three console wedges during large serial transfers shared one state: the interrupt enabled and pending, but never delivered. Why delivery is lost is still not understood, but it is no longer fatal — the handler now drains both directions and the 5 ms tick is a full polled rescue, so the console degrades to polled instead of dying.
- **`vi`.** The editor has always been on the card, but only under its own name, `levee`. It now answers to `vi` as well: one file, two names, no extra space.
- **`mmedit`'s function keys work from a serial terminal.** They always worked from a USB keyboard on the HDMI console, but a terminal such as TeraTerm spells F1–F4 differently — `ESC [ 11 ~` where the console sends `ESC O P` — and the editor silently ignored the spelling it did not know. Since the editor is driven almost entirely by function keys, that made it close to unusable over the serial port. It now accepts every sequence MMBasic's own `MMInkey` accepts, including the shifted forms.
- **`INKEY$` returns MMBasic's key codes.** A terminal sends an arrow or a function key as a several-byte escape sequence; MMBasic hands back one code for it, and until now a translated program here saw the raw bytes one at a time instead. So `IF INKEY$ = CHR$(&H80)` — the way every PicoMite program tests for cursor-up — quietly never matched. `INKEY$` now reassembles the sequence and returns the same single code the PicoMite does: `&H80`–`&H83` for the cursor keys, `&H91`–`&H9C` for F1–F12, and the editing cluster besides. Anything it does not recognise is handed back byte by byte, as before, so nothing is swallowed.
- **`vi` can edit a real file.** Levee shipped with a 4 KB buffer — a fair share of an 8-bit micro, and not of this machine. It is now 64 KB.

1.5 New in v0.8

This release gives the machine music, pins, and MMBasic's own text.

- **MP3 playback.** `PLAY MP3 f$` plays a file *while your program carries on running* — the decoder is a separate process feeding a deep buffer in the kernel, so nothing has to be refilled from a main loop. `PLAY VOLUME n` sets the level and is remembered; `PLAY STOP` stops whatever is playing, including a player left behind by a program you interrupted. A translated BASIC benchmark still runs at **89% of full speed** with music playing.
- **`SETPIN` and `PIN`** — all forty-eight GPIOs of the RP2350B, not the 28 an RP2040 has. `SETPIN n, DIN|DOUT|AIN|ARAW|OFF`, `PIN(n) = v` and the `PIN(n)` function, with analogue in volts or

raw counts. Pins are owned, and come back reset when your program ends; and a pin read or write is a register access, not a system call.

- **TEXT, FONT and all nine of MMBasic's fonts**, held in flash so they cost no RAM at all. `TEXT` takes the full alignment and scale arguments.
- **MAP, statement and function**, with `CLS [colour]`: the sixteen colours can be remapped exactly as in MMBasic, and the defaults are now MMBasic's own HDMI values rather than an approximation.
- **PAUSE gives the CPU up** instead of spinning, so a program that waits no longer holds the machine — and the fraction of a second it cannot sleep is bounded at 99 ms however long the pause.
- **Programs are much smaller.** MMBasic's scratch memory is now sized per program rather than fixed, taking a translated program's static data from 61,904 bytes to 12,748.
- **Maths runs on the DCP.** `SIN`, `COS`, `EXP` and the rest are shared out of kernel flash and use the RP2350's double-precision co-processor: about **2.7× faster** than the copy that used to be linked into every program, and it costs no program memory.
- **A C library fix that reaches everything.** `<limits.h>` declared `INT_MAX` as 32767 on a machine whose `int` is 32 bits. Any program that compared against it — or sized a buffer with it — was working from a number two thousand times too small. Every binary on the card has been rebuilt.

The kernel and the card are a matched pair, as always: the card's programs are statically linked, so a new kernel with an old card runs the old C library.

1.6 New in v0.7

Most of the kernel now executes from flash instead of being copied into RAM at boot, and the memory that frees goes to programs.

- **340 KB of program RAM**, up from 312, and a single process can have about **292 KB**. 90,676 bytes of kernel code used to be copied into RAM at boot; 45,000 of them are gone, and what is left has room to grow. Costs nothing measurable: the display DMAs out of RAM continuously, so RAM bandwidth is contended while the flash cache is a separate path.
- **64 processes**, up from 30, with the open-file and inode tables sized to match — the shell gives every background job its own `/dev/null`, so the file table ran dry before the process table did.
- `FRAMEBUFFER` — `CREATE`, `WRITE N|F`, `COPY s,d[,B]`, `CLOSE`, `WAIT`. Draw off-screen and show the frame in one go. A redraw-and-show frame costs about 4.4 ms against the 17 ms the display gives you, and `COPY ...,B` holds a loop to the refresh rate.
- `PRINT @(x, y [, mode])` puts text at a pixel position. In a graphics mode `PRINT` now *draws* the characters, as MMBasic does, rather than sending them to the console — so text follows the drawing into the framebuffer instead of the console scrolling the display out from under it. `mode` 1 draws over what is there, 2 swaps ink and paper.
- `INKEY$` — which was not implemented at all, and because it ends in `$` was quietly being treated as an ordinary empty string variable, so `LOOP UNTIL INKEY$ <> ""` could never exit.
- `PIXEL x(), y(), c()` — the array form, one call for a whole run of points instead of one syscall each.
- **Text scrolls the same way everywhere.** A `PRINT` on the last line scrolls in a graphics mode exactly as it does on the console, because it is now literally the same code.
- **A stretched ellipse outline is no longer dotted** — the fix from MMBasic's own `DrawCircle`.

- **`PRINT "x";` reaches the screen.** A partial line used to sit in a buffer until the next newline, so a program printing "Calculating. . . " and then working for half a minute showed nothing until it finished.
- **BBC BASIC works again.** It sizes its workspace by asking for memory until the kernel refuses; v0.6 let it take every last block, leaving the shell nowhere to return to. A process may now grow only as far as leaves room for the largest other one.
- **The boot banner names the PC3 release**, alongside Fuzix's own version — they are different numbers, and only one of them used to be on screen.

1.7 New in v0.6

This release is about memory. A translated BASIC program could not hold a framebuffer-sized array, and a large one could not be loaded at all; both are fixed, and the same work made everything faster.

- **BASIC arrays and strings live in the PSRAM heap.** They used to sit in the 48 KB the interpreter gave a program, so a 38,400 byte array — one MODE 1 framebuffer — did not fit at all. Globals go in one block taken once; `LOCAL` arrays and strings get a block per invocation, freed on the way out, which is what makes recursion correct. Simple variables stay where they are: they are the hot ones, and SRAM is 3.7× faster to reach than PSRAM.
- **Programs are 15% faster.** A program address is now a machine address rather than an offset into a 48 KB window, so every access stops paying for the conversion. The solar eclipse went from 3.31 to 2.80 seconds, and the heap change above costs about half a percent on top.
- **A process may use all of program RAM.** The old 256 KB ceiling was the size of a fixed swap slot, and swap has not worked that way for a while.
- **`SYSTEM`, `SAVE IMAGE` and `LOAD IMAGE` work from large programs again.** `fork` needed the process resident twice over, so anything past half of memory could not do it. The parent is now staged into PSRAM instead.
- **A memory leak fixed** that cost 132 KB — of 312 KB — every time a program failed to start, for the rest of that boot.
- **A C library bug fixed** that made `fread` and `fseek` disagree about a file's position and quietly return the wrong bytes. It needed a file small enough to fit one buffer, which is why it went unnoticed: it took down the compiler on a 305-byte object while 100 KB ones were fine. It is not specific to this machine and has been reported upstream.

1.8 New in v0.5

- **MMBasic draws.** `MODE`, `COLOUR`, `PIXEL`, `LINE`, `CIRCLE` and `RGB()` are translated and run on the HDMI display — a whole shape crosses into the kernel in one call, so a 640-point line costs one syscall rather than 640.
- **`SAVE IMAGE` and `LOAD IMAGE`**, with MMBasic's BMP decoder: 1/4/8/16/24/32-bit files, RLE4 and RLE8, and its dithering modes.
- **`SYSTEM`** — a BASIC program can run another program and wait for it, which is how `SAVE IMAGE` and `LOAD IMAGE` are built.
- **`mmedit`**, MMBasic's editor, ported.
- **`TIMER` is a float** off a 64-bit microsecond clock, and **`RND` returns 53 bits** from splitmix64 rather than 15 from the C library.
- The SD root filesystem is now **built from source** by `mksdimage.sh` — it had no build recipe before — and is **64000 blocks** rather than the 65535 that sat on `blkno_t`'s ceiling.

- A **filesystem corruption fixed** that had been destroying cards under repeated large-file rewrites: `f_trunc` freed a file's double indirect block and left the inode still pointing at it. It bit any file over 273 blocks. See the release notes.
- **Panic messages now reach the serial port.** They were being truncated to about eleven characters by an interrupt-driven UART that stopped with the machine.

2 Installing Fuzix

2.1 Flashing the kernel

The kernel is a standard UF2 file, `fuzix.uf2`, flashed exactly like any other Pico Computer firmware:

1. Set the USB HUB switch to **DISABLE** and connect the **Prog** port to your PC.
2. Hold **BOOT**, click **RESET**, release BOOT. A USB drive appears.
3. Drag `fuzix.uf2` onto the drive. The board reboots when the copy completes.
4. Set the HUB switch back to **ENABLE**.

Flashing does not touch the SD card.

2.2 Writing the SD card

Fuzix boots from a ready-made card image, `pc3-sd.img` (978 MB, about 4.9 MB compressed for download). Write it to a card of **1 GB or larger** with any raw-image tool — Raspberry Pi Imager ("use custom image"), Win32DiskImager, balenaEtcher, or `dd` on Linux/macOS. **The whole card is overwritten.**

The image lays the card out as three partitions:

Partition	Size	Type	Purpose
1	128 MB	FAT	File interchange with Windows/macOS/Linux
2	800 MB	Fuzix root	The Unix filesystem (boot device `hdb2`)
3	4 MB	0x7F	Reserved

The root filesystem is 1,638,400 blocks of 512 bytes with 25,600 inodes, and it fills its partition exactly.

It did not always. Before v0.9 a block number was 16 bits, so 65535 was at once the highest block a filesystem could have and the marker meaning "no such block" — and the filesystem was deliberately held to 64000 blocks, short of its own partition, so that a stray value could be recognised as wrong instead of pointing at a real sector. That is what limited the root to 32 MB. From v0.9 the format is **FS32**: 32-bit block numbers and 256-byte inodes, with a "no such block" marker that cannot be a real block at all. The margin is gone, and with it the ceiling — a filesystem can now be up to 2 TB and a single file up to 1 GB.

A v0.9 kernel and a v0.9 card go together. The two formats are not interchangeable and neither pretends otherwise: a v0.9 kernel refuses an older card by name at the `bootdev:` prompt, and an older kernel refuses a v0.9 one. Flash `fuzix.uf2` and write the card in the same sitting.

Since v0.5 the filesystem has been built from source by `mksdimage.sh`, so the card can be reproduced rather than merely copied. `mkcard.sh` decides the layout, and its `FAT_MB` and `ROOT_MB` settings are the only place the partition sizes are written down.

Partition 1 ships unformatted: format it as **FAT or FAT32 (not exFAT)** in Windows before first use — see section 6. Partition 2 is Fuzix's own filesystem format; no desktop OS can mount it, which is why the FAT partition and the `fat` command exist.

2.3 First boot

Connect a monitor to the HDMI port and/or a terminal program (for example TeraTerm at 115200) to the USB-C console port. Switch on: the kernel banner, board identification and device probe appear on both, then:

```
login: root
```

There is no password. The system arrives at a Bourne shell; the console is an 80×40 colour terminal (termcap type `pc3`) and the USB keyboard and serial input feed the same session interchangeably. Set the keyboard layout in `/etc/rc` (`picoctl keymap uk` by default).

To power off, type `shutdown` (or at minimum `sync`, then wait a moment). After an unclean power-off the next boot repairs the filesystem automatically (`fsck -a -y`).

2.4 Setting the clock

The board's DS3231 battery-backed clock supplies the date and time: at every boot `/etc/rc` runs `setdate`, which reads the chip and sets system time silently. If the chip cannot supply a valid time (fresh battery, or the battery was removed) the same command falls back to prompting on the console:

```
Current date is Mon 2026-07-27
Enter new date:
```

Press Enter to keep a value, or type a new one (`2026-07-28`, then `16:30:00`). To set the clock at any other time run `setdate -u`, which always prompts.

Setting the system time does not touch the chip. To make the time permanent, write it back:

```
# setdate -w
writing
```

That also restarts a stopped oscillator, so the sequence for a new battery is: `setdate -u` (enter the time), then `setdate -w`. The kernel message `oscillator was stopped, time needs setting` at boot means exactly that sequence is wanted.

While running, system time is kept by the crystal-driven timer tick and gently re-synchronised from the DS3231 about once an hour, the same policy as MMBasic. `date` prints the current time.

If a transaction to the chip is ever interrupted at the wrong moment (a reset mid-read), the battery-backed DS3231 can be left jamming the I2C bus - even across power cycles. The kernel detects this at boot (`ds3231: SDA held low, clocking bus free`) and clears it automatically.

2.5 Command history and line editing

At any cooked-mode prompt - the shell, login, most line-oriented programs - the console provides in-line editing and command history:

Key	Action
Up / Down	walk back / forward through previous commands
Left / Right	move within the line
Home / End (or Ctrl-A / Ctrl-E)	start / end of line

Key	Action
Backspace / Delete	delete left / at the cursor
Ctrl-U	discard the line

The history (about 500 commands) lives in a small region reserved at the top of the PSRAM, so it costs no program memory; like the RAM disc it survives a reset but not a power cycle. Programs that take the terminal raw (BBC BASIC, the editor) see every keystroke themselves as usual - BBC BASIC has its own line editor and *EDIT.

2.6 Escape routes

- **Esc** — inside BBC BASIC, stops the running program.
- **Ctrl-** — kernel emergency break: kills the foreground program even if it has the terminal in raw mode, and restores a sane terminal. Use it instead of the RESET button when something wedges.
- `kill` from the shell works on anything; Fuzix pre-emption guarantees a spinning process can't lock you out.

3 The consoles

Everything is mirrored: kernel and program output appear on the HDMI display and the serial console simultaneously, and input is merged from the USB keyboard and the serial line. TeraTerm is resized to 80×40 automatically at boot.

While BBC BASIC has a graphics MODE on screen, the HDMI display belongs to the graphics; text output continues to the serial side as a plain stream, so a transcript survives even a full-screen game.

4 BBC BASIC

Start it by name; leave with `QUIT`:

```
# bbcbasic
BBC BASIC for Fuzix Console v0.50
(C) Copyright R. T. Russell, 2025
>
```

This is R. T. Russell's BBC BASIC (the BBCSDL console edition), so the language is the full modern dialect: 64-bit integers, IEEE double floats, long variable names, `WHILE`/`ENDWHILE`, multi-line `IF`/`ELSE`/`ENDIF`, structures, and the inline assembler. Keywords must be typed in **capitals** (`PRINT`, not `print`); `*LOWERCASE ON` relaxes this. Star commands are case-insensitive.

4.1 Loading programs, including plain text

`LOAD` and `CHAIN` accept **both** program formats:

- the tokenised internal format written by `SAVE` (fast, exact), and
- **plain ASCII listings**, detected automatically. An unnumbered modern-style listing is given line numbers 10, 20, 30. . . ; a listing with its own numbers keeps them. Windows line endings are fine, and typographic characters that desktop editors introduce (curly quotes, non-breaking spaces, tabs) are silently converted to their ASCII equivalents — an invisible "smart" character can otherwise produce a baffling `Syntax error` on a line that looks perfect.

The comfortable round trip: edit `myprog.bas` on your PC, copy it to the FAT partition, then:

```
# fat get myprog.bas
# bbcbasic
>LOAD "myprog.bas"
>RUN
>SAVE "MYPROG"          save tokenised for fast loading next time
```

4.2 Graphics

`MODE 0`–`5` switch the display to 1024×768 and provide the classic screens; `MODE` with any higher number (e.g. `MODE 6`) returns to the text console. Errors leave you at the prompt *in* the current mode, exactly like the original machine.

MODE	Resolution	Colours	Presentation on the monitor
0, 3	640×256	2	Full width (5:8 smooth upscale), full height
1, 4	320×256	4	960×768 — every pixel a crisp 3×3 block
2, 5	160×256	16	960×768 — every pixel a crisp 6×3 block

Coordinates are the authentic 1280×1024 graphics units, origin bottom-left, movable with `VDU 29`. Implemented: `MOVE`, `DRAW`, `PLOT` (move/line families 0–15, points 64–71, filled triangles 80–87 — so `PLOT 85` works), `CLG`, `GCOL`, `COLOUR`, `POINT(x,y)`, `VDU 19` palette changes (instant, no redraw), `TAB(x,y)`, and text printed with the built-in 8×8 font (40 or 80 columns × 32 rows, with scrolling). `MODE 1/4` store 4 bits per pixel, so all 16 logical colours are available there via `VDU 19` even though the default palette is the authentic four.

4.3 Sound

The full BBC sound system plays through the PCM5102 DAC and the 3.5 mm jack:

```
SOUND 1,-15,89,20                          A4 (440 Hz) for one second
ENVELOPE 1,1,4,-4,4,10,10,10,126,-2,0,-10,120,100
SOUND 1,1,100,30                           envelope-shaped note
SOUND &101,-15,53,20:SOUND &102,-15,69,20:SOUND &103,-15,81,20
                                           a synchronised C-E-G chord
```

Channels 1–3 are square-wave tones, channel 0 is noise. Each channel queues up to 8 notes; `&1x` in the channel number flushes the queue, `&1xx`–`&3xx` synchronise channels. `ENVELOPE` implements the three-section pitch envelope and ADSR amplitude, stepped at the authentic 100 Hz. Pitch follows the BBC scale: 4 units per semitone, 89 = A4 = 440 Hz. Durations are in 20ths of a second; 255 means "until further notice". `SOUND` blocks BBC-style when a queue is full (Esc still works).

There is one audio output, so the synthesiser and MP3 playback are mutually exclusive — as they are in MMBasic. While a track is playing, notes are queued and heard when it ends; `PLAY STOP`, or `killing playmp3`, gives the synthesiser the hardware back at once.

4.4 ADVAL: joystick, analogue inputs, sound queues

Call	Returns
`ADVAL(0)`	Joystick switches on GP34–37 as a mask: bit 0 = GP34, bit 1 = GP35, bit 2 = GP36, bit 3 = GP37; pressed (grounded) = 1
`ADVAL(1)`–`ADVAL(4)`	Analogue readings of GP41–GP44, 0–65520 (12-bit ADC ×16)
`ADVAL(-1)`	Characters waiting in the keyboard buffer
`ADVAL(-5)`–`ADVAL(-8)`	Free queue slots on sound channels 0–3
`ADVAL(-9)`	Hardware microsecond counter (64-bit)

The joystick inputs have pull-ups and Schmitt-trigger inputs enabled; wire switches directly to ground. The analogue inputs are 3.3 V full-scale.

`ADVAL(-9)` is the benchmarking clock — the RP2350's free-running microsecond timer, far finer than `TIME`'s centiseconds (which tick in tenths of a second on this kernel). MMBasic-style usage:

```
DEF FNtimer = ADVAL(-9)
T% = FNtimer
REM ... code under test ...
PRINT (FNtimer - T%) / 1000; " ms"
```

Each reading costs one system call (a few tens of microseconds) - negligible over millisecond-scale measurements. The value is the full 64-bit hardware counter (BBC BASIC integers are 64-bit), counting microseconds since power-on: it never wraps in practice.

4.5 Serial port

A second serial port lives on the I/O header: **TX = GP0, RX = GP1** (3.3 V logic, no flow control), visible as `/dev/tty2`. BBC BASIC reaches it as a *port channel* — raw and unbuffered:

```
*stty 9600 </dev/tty2
ch% = OPENUP("/dev/tty2")
BPUT#ch%,65                 send a byte
X% = BGET#ch%               receive a byte (waits for one)
CLOSE#ch%
```

Any `stty` setting applies (baud rate, parity, size). The same `OPENIN`/`OPENUP`/`BGET#`/`BPUT#` mechanism works on any `/dev` device; ordinary file names get the buffered 256-byte file channels instead, exactly as on the original machine.

4.6 Star commands and the shell

The built-in set includes `*DIR`/`*.`, `*CD`, `*TYPE`, `*COPY`, `*DELETE`/`*ERA`, `*MKDIR`, `*RMDIR`, `*KEY`, `*SPOOL`, `*EXEC`, `*LOWERCASE`, `*TEMPO`, `*QUIT`. Anything unrecognised is passed to the Unix shell, so `*ls -l`, `*stty 9600 </dev/tty2` and even `*fat get demo.bas` work from inside BASIC.

4.7 The inline assembler

The assembler between `[` and `]` targets the machine it runs on: **ARM Thumb (v6-M subset)**, not 6502. `CALL` and `USR` work as documented for BBCSDL's ARM editions. Machine code written for a BBC Micro will not run; this is the same situation as every modern BBC BASIC.

4.8 Differences from the original BBC Micro

Better than the original:

- ~110 KB of program workspace (about 80 KB when graphics are in use) against the Model B's 32 KB shared with the screen
- 64-bit integers, double-precision floats, the modern language
- Long file names on a hierarchical filesystem
- The machine multitasks underneath — BASIC is just a process

Not (yet) present, compared with an original machine or full BBCSDL:

- **MODE 7** teletext (MODE 6 and 7 return to the console)
- `VDU 5` text-at-the-graphics-cursor; user-defined characters (`VDU 23`) are accepted but not rendered
- PLOT families beyond points/lines/triangles: no circles, arcs, ellipses, flood fill or rectangle/parallelogram fills yet
- `GCOL` action modes 1–4 (OR/AND/EOR/invert) plot as mode 0 (set)
- Flashing colours (8–15 map to their steady equivalents)
- `*FX` calls, `INKEY` with negative arguments (keyboard scanning), and the 6502 `CALL` interface
- `TIME` advances in 0.1 s steps (the kernel clock granularity)
- Cassette/Econet/ROM filing systems, for obvious reasons

5 The C compiler

The machine compiles C on its own, with no PC involved. `cc` is Alan Cox's Fuzix Compiler Kit, retargeted here to a bytecode machine.

```
# cat > hello.c
#include <stdio.h>

int main(void)
{
        printf("hello from Fuzix\n");
        return 0;
}
^D
# cc hello.c
# ./hello.bc
hello from Fuzix
```

`cc prog.c` writes `prog.bc` and makes it executable, so `./prog.bc` runs it directly — the object starts with `#!/usr/bin/bcrun`, and the kernel does the rest. `bcrun prog.bc` still works and is the way to run an object that has lost its execute bit. Options are `-o name`, `-v` to show each pass as it runs, and `-k` to keep the intermediates. `bcdump prog.bc` disassembles.

`cc` is a driver: it runs `cpp`, then the three compiler passes from `/usr/lib/cc`. The passes cannot be driven from the shell by hand — two of them read back from their own standard output, which needs a redirection the Bourne shell here does not have. Like BBC BASIC, the compiler lives on the SD card root and is not in the NAND recovery system.

A small program takes about a second to compile. The dots `cc` prints are progress, one per top-level declaration.

5.1 What it compiles

C89, plus declarations after statements — the one C99 borrowing, because refusing it is a nuisance out of proportion to the standard it comes from. Everything else is ANSI C as of 1989: the full type system, `struct` and `union` including passing and returning them by value, `enum`, `typedef`, function pointers, `switch`, `goto`, all the usual operators, 64-bit `long long`, and double-precision floating point.

The compiler is checked against gcc as an oracle: 165 of the 175 applicable tests in the public `c-testsuite` conformance suite pass with byte-identical output, and a set of samples in the source tree is diffed against gcc on every change — on the board as well as on the host.

5.2 Headers and the runtime library

`/usr/lib/cc/include` holds `stdio.h`, `stdlib.h`, `string.h`, `math.h`, `assert.h`, `limits.h` and `stddef.h`. These describe what `bcrun` actually provides, and they are deliberately **not** `/usr/include`, which describes the Fuzix C library used by native binaries.

Available: `printf sprintf puts putchar`; `fopen fclose fread fwrite fgetc getc fputc putc fgets fputs fprintf feof fseek ftell rewind fflush remove`; `malloc calloc realloc free exit atoi abs`; `strlen strcpy strncpy strcat strcmp strncmp strchr strrchr memset memcpy` memmove memcmp; and from math.h, in double precision, sin cos tan asin acos atan atan2 sinh cosh tanh sqrt exp log log10 pow floor ceil fabs fmod. Each of those is a native function inside bcrun, so it runs at machine speed whatever the calling code is.

malloc takes its memory from the PSRAM, not from the program's own 340 KB, so a C program can allocate far more than it could hold itself. It is asked for once when the program loads; set BCRUN_HEAP to a size in KB in the environment to change how much.

The raw system calls open creat close read write lseek unlink are also linked in, but no header declares them — declare them yourself if you want them. Two extras reach the hardware: time_us() returns the microsecond counter, and adval(n) is the BASIC ADVAL function (joystick, analogue inputs, sound queues).

5.3 Limitations

These are worth knowing before you start a large program.

- **One source file per program.** There is no assembler and no linker: cc2 writes a loadable object directly, so nothing can be linked together. cc rejects a second source file rather than pretending.
- **Bitfields are not implemented.** unsigned flags : 3; is refused with a diagnostic. This is the only part of C89 missing.
- **& on a library function does not work.** A runtime function is resolved by index and has no address, so &printf cannot be represented. bcrun refuses such a program by name when it loads it, rather than running it with something wrong in place of an address. Pointers to *your own* functions are entirely fine.
- **No wide strings.** L'x' is accepted (and equals 'x' here); L"..." is refused rather than quietly returned as a narrow array.
- **printf rounds halves up.** %.2f of 0.125 gives 0.13 where a full C library gives 0.12. A deliberate, documented difference: matching the round-half-to-even rule needs exact decimal expansion.

5.4 Speed

cc2 translates each function it compiles into native Thumb-2 for the Cortex-M33 and stores that in the object beside the bytecode; bcrun runs the native code and keeps the interpreter for whatever did not translate. Nothing needs to be asked for — it is simply what the compiler does.

Dhrystone 2.1, compiled on the machine, runs at about **167,000 Dhrystones/second** — 44% of the same benchmark cross-compiled by gcc -O2 for the same chip (379,000). Individual loops do better: a sieve, a shell sort and an xorshift generator all land within a factor of two or three of gcc, and double-precision arithmetic uses the RP2350's hardware co-processor through the same routines the rest of the system uses.

That figure has moved a long way and is still moving: the same benchmark ran at 3,808/second on the original pure interpreter and 90,000 as recently as v0.8. Most of the recent ground came from keeping the hot code in native form rather than from generating better instructions — binding every library symbol once at load, caching the hottest local in a register, and inlining the compound assignments (i += n) on the 64-bit and double types that every MMBasic loop counter uses. Each of those had been quietly leaving native code and crossing back into the C runtime once per iteration.

A program can be forced to interpret with `BCRUN_BYTECODE=1` in the environment, which is how the native code is checked against the interpreter — the two must agree exactly.

What the compiler is *for* is being able to write and build real programs on the machine itself — utilities, file handling, glue, and now whole applications where the speed matters. Nothing else on the Pico Computer can do it: this is a complete toolchain that runs on the hardware, not a cross-compiler.

The bytecode is also a deliberately language-neutral intermediate form, so front ends for other languages can share the same back end and runtime. `mmbc`, the MMBasic translator described next, is the first.

6 MMBasic: the `mmbc` translator

`mmbc` translates an MMBasic program into C. `cc` compiles that C into native ARM code. The result runs as an ordinary program, several times faster than the same source under the MMBasic interpreter, on the same machine at the same clock.

Nothing about it is a cross-development trick: both programs live on the SD card and run on the Pico Computer.

6.1 The three commands

```
# mmbc prog.bas          -> prog.c
# cc prog.c              -> prog.bc
# ./prog.bc              runs it
```

`mmbc` writes `prog.c` next to the source and says so. `-o name.c` puts it somewhere else. Every line it cannot translate is reported with its line number and left as a comment in the C, so a program that uses something unsupported still produces a translation you can read, and tells you exactly where it gave up.

On the machine, `mmbc` writes C for the machine's own compiler. On a host it writes the slightly different C that gcc prefers; `--fcc` and `--gcc` force either form, which only matters if you are moving the generated C between the two.

6.2 A first program

```
# cat > hello.bas
Print "Hello from MMBasic"
For i = 1 To 5
  Print i, i * i
Next i
^D
# mmbc hello.bas
wrote hello.c
# cc hello.c
....
# ./hello.bc
Hello from MMBasic
 1       1
 2       4
 3       9
 4       16
 5       25
```

The dots from `cc` are one per top-level declaration — a translated program carries the MMBasic runtime's declarations with it, so there are a couple of hundred of them even for four lines of BASIC. That is also why the first compile takes longer than the size of your program suggests: about half a minute here, and much the same for anything small.

`Print` with a comma gives MMBasic's tabbed columns, as above.

6.3 A worked example: the KnivD benchmark

This is a published MMBasic benchmark, unmodified. It runs four thirty-second rounds and reports a score.

```
# cat bench.bas
Print "MMBASIC benchmark (C) KnivD 2016"
Dim integer t, i=0
Dim float x(1000), f=0.0
Dim string s=""
Print "Calculating... ";
count%=0
Do
  s=""
  f=0.0
  i=0
  Timer =0
  Do While Timer<30000
    i=i+2 : f=f+2.0002
    If (i Mod 2)=0 Then
      i=i*2 : i=i\2
      f=f*2.0002 : f=f/2.0002
    End If
    i=i-1
    For t=1 To 100
      f=f-1.0001
      If (f-Int(f))>=0.5 Then
        f=Sin(f*Log(i))
        s=Str$(f,6,6)
      End If
      f=(f-Tan(i))*(Rnd(0)/i)
      If Instr(s,LEFT$(Str$(i),2))>0 Then s=s+"0"
    Next
    x(1+(i Mod 1000))=f
  Loop
  Print Chr$(13)+"Performance: "+Str$((i*1024)\286,8,0)+" grains"
  Inc count%
Loop Until count%=4
End

# mmbc bench.bas
wrote bench.c
# cc bench.c
......................................................................
# ./bench.bc
MMBASIC benchmark (C) KnivD 2016
Calculating... Performance:    28944 grains
Performance:    28944 grains
Performance:    28944 grains
```

```
Performance:    28940 grains
```

MMBasic itself scores about 12,000 grains on the same board at the same clock. Note what the program exercises: 64-bit integers, doubles, string building, `Str$`, `Instr`, `Sin`, `Log`, `Tan`, `Rnd`, a thousand-element array and a timer — all of it translated, compiled and running natively.

6.4 A worked example: something numerical

A longer one. `solar_eclipse.bas`, in `/root/cc`, is a 3,200-line astronomical calculation (Bessel elements, lunar and solar series) that reads a date and prints the circumstances of an eclipse. It is a good test because every digit of its output can be checked against other machines.

```
# mmbc solar_eclipse.bas
wrote solar_eclipse.c
# ./solar_eclipse.bc < solar_eclipse.in
...
event duration          2.61100904 hours
Time taken :     2.803  Seconds
```

The same program takes 12.5 seconds under MMBasic and 8.8 seconds under MicroPython on this hardware; every digit printed is identical in all three. Translating it takes a few seconds. Compiling it is a much longer job than the benchmark above — it is a hundred and forty kilobytes of C — so start it when you have the machine to yourself.

6.5 What the translation looks like

The C is meant to be read. Variables keep their names, control flow keeps its shape, and the MMBasic semantics that C does not share — integer division, `Mod` on negative numbers, string handling, the 64-bit integer/double type rules — are handled by a small runtime inside `bcrun` rather than by rewriting your program into something unrecognisable.

```
# mmbc hello.bas
# head -20 hello.c
```

is worth doing once, to see what your program became.

6.6 Strings, arrays and functions

Strings are MMBasic strings: `Dim string s` gives the usual 255 characters, and `Dim string s length 40` the shorter form. Arrays index from `OPTION BASE` as they should, and `Bound()` reports their limits. `Sub` and `Function` work, including arrays passed by reference and modified in place.

Arrays and strings live in the PSRAM heap, so what limits them is the 8 MB rather than the size of a process. A framebuffer-sized array — `Dim Integer fb(4800)`, 38,408 bytes — is unremarkable now; before v0.6 it would not fit at all. Simple variables stay in the program's own memory, where they are quicker to reach, which is the same split MMBasic itself makes and for the same reason.

`LOCAL` arrays and strings get their own block for each call, released when the routine returns, so a recursive routine sees its own copy at every level. `STATIC` locals are unaffected — they are meant to outlive the call, so they stay where they were.

The one thing to remember is that a translated program is a *program*, not an interactive session: there is no editor, no `RUN`, no immediate mode. You edit the `.bas` file, translate, compile and run —

which is also why a translated program can be put in `/usr/bin` and used like any other command.

6.7 Errors

`mmbc` reports what it could not translate, by line, and carries on:

```
# mmbc gpio.bas
2 line(s) could not be translated and were commented out:
  line 2: expected '='
  line 3: 'pin' is not an array
wrote gpio.c
```

The reasons are the translator's own — it reads `SetPin GP1, DOUT` as far as it can and then says what it wanted to see — so they name the symptom rather than the statement. The lines themselves appear in the C as comments, with the original text, which is the quickest way to see what was dropped:

```
    /*     SetPin GP1 , DOUT */
```

The translation still happens and everything else works. `--strict` stops on the first such line instead, and `--report` lists them again at the end along with any implied global variables.

Appendix C lists what is covered.

6.8 Graphics

A translated program draws on the HDMI display with MMBasic's own statements:

```
MODE 2
COLOUR RGB(YELLOW)
LINE 0, 0, 319, 239
CIRCLE 160, 120, 60, , , RGB(WHITE), RGB(BLUE)
PIXEL 10, 10, RGB(RED)
```

Two modes are available, chosen to match the VGA builds of MMBasic and the first two HDMI modes:

`MODE`	Resolution	Colours
`MODE 1`	640×480	monochrome
`MODE 2`	320×240	16, from MMBasic's RGB121 set

`RGB()` takes the usual named colours and `RGB(r,g,b)`. `COLOUR` sets the current drawing colour, which every statement uses when no colour is given. `LINE`, `CIRCLE` and the rest take MMBasic's argument order, including the blank arguments — `CIRCLE x, y, r, , , fill` is written exactly as MMBasic writes it.

The drawing primitives live in headers as static functions, one header per primitive — `mmb_gfx_circle.h`, `mmb_gfx_box.h`, `mmb_gfx_rbox.h`, `mmb_gfx_triangle.h`, `mmb_gfx_arc.h`, `mmb_gfx_text.h`, `mmb_gfx_map.h`, over the shared batch helpers in `mmb_gfx_pts.h` — and the translator includes exactly the ones the program uses, so a program that never draws a circle does not carry the circle code. (`cc` discards a file-scope static nothing calls, but that rule counts names rather than reachability, so a recursive primitive survives inside any header that carries it — which is why the include, not the static, is the unit that matters.) And a whole shape crosses into the kernel in a single call, so a 640-point line is one syscall, not 640 — measured at 71 µs against 433 µs for the naive version.

The geometry is MMBasic's, copied rather than re-derived, down to the dotted appearance of a stretched ellipse. If a picture differs from MMBasic's, that is a bug worth reporting.

One caution. In `MODE 1` the console and the graphics share a single framebuffer, so anything printed — including the cursor — appears on the picture. It is visible in a `SAVE IMAGE` of the whole screen as a difference in the top text line. Restricting the saved region avoids it; directing program output elsewhere is planned.

6.9 The sixteen colours: `MAP`

`MODE 2` stores a colour *number* per pixel, 0 to 15, and `MAP` says what colour each number actually is. Change entry 8 and everything already drawn in colour 8 changes with it — without redrawing a single pixel. That is what makes fades, flashes and colour cycling free.

```
MAP(4) = RGB(255, 128, 0)      ' collect
MAP SET                        ' ... and apply, all at once
```

`MAP(n) = colour`	set entry `n`, 0 to 15. **Nothing changes yet**
`MAP SET`	apply the collected palette, during the frame blanking
`MAP RESET`	back to the mode's own sixteen
`MAP MAXIMITE`	the original Colour Maximite's sixteen
`MAP GRAYSCALE`	sixteen greys, 0 to 255 in steps of 17 (`GREYSCALE` too)

The two-step is the point, and it is MMBasic's own: applying a new palette one entry at a time shows the picture half recoloured, and `MAP SET` waits for the blanking so the change lands between frames.

`MAP(n)` is also a **function**, giving the colour entry `n` stands for by default — which is the colour a program must ask for to land on that entry:

```
FOR i = 0 TO 15
  LINE i * 20, 0, i * 20, 199, , MAP(i)     ' one line per entry
NEXT i
```

It reports the default and takes no notice of any remapping, exactly as MMBasic does; it is the inverse of the fixed rule that turns a colour into an entry number. That rule is bit extraction — one bit of red, two of green, one of blue — and it does **not** depend on the palette, so remapping never moves where new drawing lands.

`MAP` needs a 16-colour mode; in `MODE 1` it is an error.

6.10 Text on the picture: `TEXT` and `FONT`

`TEXT` puts a string at a pixel position, justified how you ask, in any of nine fonts:

```
TEXT 160, 120, "Centred", "CM"
TEXT 319, 0, "top right", "RT", 3
TEXT 0, 100, "big and red", "LT", 5, 2, RGB(RED)
TEXT 0, 200, "over the picture", "LT", 1, 1, RGB(WHITE), -1
```

```
TEXT x, y, string$ [, alignment$] [, font] [, scale] [, colour] [, background]
```

Everything after the string may be left out, and a blank argument takes the default, as everywhere in MMBasic. `background` of `-1` is transparent, which is how you write over a picture without a box of paper around the letters. With no colours given, `COLOUR`'s are used.

`alignment$` is up to three letters and any of them may be omitted:

	letters	meaning
horizontal	`L C R`	x is the left, the centre, the right
vertical	`T M B`	y is the top, the middle, the bottom
orientation	`N V`	normal, or one character per line downwards

The other three orientations MMBasic offers — `I` inverted, `U` and `D` rotated — are **not yet drawn**; they are accepted and come out normal.

The nine fonts are MMBasic's own, so a character looks the same here as it does there. They live in the PC3's flash, which is why there is no cost to having all of them:

font	cell	what it is
1	8×12	the console's own — the default
2	12×20	
3	16×24	
4	10×16	
5	24×32	
6	32×50	**digits only** — a clock face, a score
7	6×8	
8	4×6	the smallest
9	8×10	

`scale` is 1 to 15 and multiplies the cell, so font 1 at scale 2 is a 16×24 character built from the 8×12 one.

A character a font does not have prints as a blank cell. With font 6 that is everything except `0` to `9`.

`FONT` sets the font `PRINT` itself uses, in a graphics mode:

```
FONT 3, 1
PRINT "large"
```

```
FONT [#]n [, scale]
```

This changes where the next line goes as well as how the letters look — a 24×32 font fills the screen in seven lines where font 1 takes nineteen — and scrolling follows it.

6.11 The glyphs themselves: `MM.INFO(FONT ADDRESS)` and `PEEK`

`TEXT` draws on the PC3's own screen. If you have hung a display off the I/O header — an ILI9341 on SPI, say — the firmware knows nothing about it, and you are drawing every pixel yourself. You would rather not also carry your own copy of a font to do it.

You do not have to. Ask where the built-in ones are:

```
a = MM.INFO(FONT ADDRESS 3)
```

That is a machine address, and on this computer a machine address is something a program can simply read. There is no MMU and no memory protection: the fonts are `const`, so they live in flash where nothing can move them, and `PEEK` reaches them where they lie.

The first four bytes at that address are the font describing itself:

offset	
0	width in pixels
1	height in pixels
2	code of the first character it has
3	how many characters

so having the address you need nothing else — no table of sizes, no constants of your own to get wrong:

```
w     = PEEK(BYTE a)
h     = PEEK(BYTE a + 1)
first = PEEK(BYTE a + 2)
count = PEEK(BYTE a + 3)
```

The glyphs follow, each `width × height` bits, packed continuously with no padding between rows and no padding between characters, most significant bit first. The one for character c starts at

```
a + 4 + (c - first) * width * height / 8
```

and bit $y \times width + x$ of it is the pixel at (x, y). This is MMBasic's own layout, unchanged, because these are MMBasic's own nine fonts — the ones the console draws from.

Reading one back is the clearest way to see it. `fontaddr.bas` in `/root/cc` does exactly this, and prints:

```
font  address     cell   first  count
 1    1000EF10    8x12     32    224
 2    1000F994   12x20     32     95
 3    100104BC   16x24     32     95
 5    10012814   24x32     32     95
 6    10014BB8   32x50     48     11
```

Font 6 saying `first 48, count 11` is the digits-only font describing itself: 48 is `"0"`, and eleven characters is `0` to `9` and one more. Nothing in the program was told that.

```
MM.INFO(FONT ADDRESS n)     n is 1 to 9; 0 if there is no such font
PEEK(BYTE addr)             one unsigned byte
```

```
PEEK(SHORT addr)            sixteen bits, signed
PEEK(WORD addr)             thirty-two bits, unsigned
PEEK(INTEGER addr)          sixty-four bits, signed
PEEK(FLOAT addr)            a double
```

Both spellings are MMBasic's, so a program written this way runs on a PicoMite unchanged.

These are sharp. A `PEEK` of a wrong address is not an error message — there is nothing on this machine to catch it. It reads whatever is there, or the program dies. Check the address you were given before you use it:

```
a = MM.INFO(FONT ADDRESS 3)
IF a = 0 THEN ERROR "no font 3"
```

The wider forms insist the address is a multiple of their width, as MMBasic does, and say `Address not divisible by 4` if it is not. That is a real check and not a formality: an unaligned load faults on this processor.

MMBasic's `PEEK(VAR ...)`, `PEEK(VARADDR ...)` and `PEEK(CFUNADDR ...)` are not translated — those ask about a *variable* rather than an address, which needs the symbol table. Neither is `POKE`, in any form. `MM.INFO` translates `FONT ADDRESS` and nothing else; the other things MMBasic's `MM.INFO` reports that make sense here already have their own spellings (`MM.HRES`, `MM.VRES`, `MM.DEVICE$`, `MM.VER`, `MM.ERRNO`).

There is a worked example of all of this driving a real panel under A worked example: a barometric station.

6.12 Animation: FRAMEBUFFER

Drawing straight to the screen means the monitor shows the picture half-finished — the erase, then each shape appearing in turn. MMBasic's answer is to build the frame off-screen and show it in one go, and it is here:

```
MODE 2
FRAMEBUFFER CREATE
FRAMEBUFFER WRITE F
DO
  CLS
  CIRCLE x, y, 20, , , 0, RGB(YELLOW)
  FRAMEBUFFER COPY F, N, B
LOOP WHILE INKEY$ = ""
```

`FRAMEBUFFER CREATE`	make the off-screen buffer, blank
`FRAMEBUFFER WRITE N | F`	send drawing to the screen, or to the buffer
`FRAMEBUFFER COPY s, d [, B]`	`s` and `d` each `N` or `F`; `B` starts at the top of the frame
`FRAMEBUFFER CLOSE [F]`	give the buffer back
`FRAMEBUFFER WAIT`	wait for the top of the frame

Everything that draws follows `WRITE`, `CLS` included — so `CLS` while writing to `F` clears the buffer and leaves the picture on the screen alone.

```
CLS [colour]
```

fills whatever is being drawn on. With no colour it uses the background from `COLOUR`, so `COLOUR RGB(WHITE), RGB(BLUE)` then `CLS` gives a blue screen; `CLS RGB(GREEN)` fills with green without changing what `COLOUR` set. Either way the text cursor goes back to the top left, as MMBasic's does.

`CREATE` belongs **after** `MODE`. A mode change throws the buffer away, because what is in it is in the geometry of the mode being left and nothing converts it. That is MMBasic's rule too. You do not have to `CLOSE`: the buffer comes back automatically when the program ends.

Only one program can hold the buffer at a time — a second one is told so rather than quietly sharing it — and only the program holding it draws into it. Anything else that draws while your program is between frames, a shell prompt included, still goes to the screen.

What it costs. The buffer is in PSRAM, so drawing into it is about 2.4× the cost of drawing on the screen — a full-screen fill measured 1.49 ms direct against 3.63 ms buffered. The copy back is 38,400 bytes at about 53 MB/s, or **0.73 ms**. So a redraw-and-show frame costs somewhere near 4.4 ms against the 17 ms the display gives you, and `COPY ... ,B` holds the loop to the refresh rate: 59 frames a second, with room for roughly three times as much drawing before one is dropped.

Not yet: `FRAMEBUFFER LAYER` and `FRAMEBUFFER MERGE`. There is one off-screen buffer and no transparent blit; both are refused by name rather than translated into something they are not.

`INKEY$` returns the key that has been pressed, or `""` if none has, without waiting — which is how the loop above ends. It leaves the terminal as it found it, so `INPUT` still works afterwards and a program stopped with Ctrl-C does not take the shell's echo with it.

A cursor or function key reaches a terminal as an escape sequence of several bytes, and `INKEY$` reassembles it and gives back the single code MMBasic gives, so the PicoMite idiom works unchanged:

```
DO
  k$ = INKEY$
  IF k$ = CHR$(&H80) THEN PRINT "up"
  IF k$ = CHR$(&H91) THEN PRINT "F1"
LOOP UNTIL k$ = "q"
```

The codes are MMBasic's: `&H80`–`&H83` for up, down, left and right, `&H84` insert, `&H86` home, `&H87` end, `&H88`/`&H89` page up and down, `&H7F` delete, and `&H91`–`&H9C` for F1 to F12. A sequence it does not recognise comes back byte by byte behind its escape, so nothing is lost — and a bare `ESC` is still `CHR$(27)`.

6.13 Running another program: SYSTEM, SAVE IMAGE, LOAD IMAGE

`SYSTEM` runs a program and waits for it:

```
SYSTEM "sum", "a.bmp", "b.bmp"
```

Each argument is passed separately, so no shell quoting is involved and a filename with a space in it needs no special care.

`SAVE IMAGE` and `LOAD IMAGE` are built on it — they run the `saveimage` and `loadimage` programs described in the applications list, which means a BASIC program that never touches an image pays nothing for them:

```
SAVE IMAGE "shot.bmp"                       ' the whole screen
SAVE IMAGE "part.bmp", 160, 120, 320, 240   ' x, y, w, h
LOAD IMAGE "shot.bmp"                       ' at 0,0
LOAD IMAGE "shot.bmp", 160, 120             ' at x,y
```

`LOAD IMAGE` takes MMBasic's full syntax including the dither mode and the source rectangle:

```
LOAD IMAGE fname$ [,x] [,y] [,mode] [,ximage] [,yimage] [,wimage] [,himage]
```

The decoder is MMBasic's, so it reads 1, 4, 8, 16, 24 and 32-bit BMPs, `BI_BITFIELDS`, and RLE4/RLE8 compression. Dithering is *optional*, as in MMBasic — omit the mode and the image is mapped without it.

6.14 Music: PLAY MP3, PLAY VOLUME, PLAY STOP

```
PLAY VOLUME 70
PLAY MP3 "/root/mp3/whiter.mp3"
' the program carries straight on, and the music plays
FOR i% = 1 TO 20
  PRINT "still working"; i%
  PAUSE 500
NEXT i%
PLAY STOP
```

`PLAY MP3` does **not** wait. The decoder is a separate program feeding a 1.5-second buffer in the kernel, and the sound hardware empties that buffer by DMA, so playback needs nothing from your program once it has started. MMBasic has to refill its audio from the interpreter's idle loop; here there is no idle loop to do it in, and no cost when there is nothing to play. A translated benchmark measured **89% of its full speed** with a track playing.

`PLAY VOLUME n` takes 0 to 100 and is remembered, so every later `PLAY MP3` uses it until it is changed again. Out-of-range values are clamped rather than refused. The default is 80. The scale is logarithmic, so 50 is a comfortable half rather than a whisper.

`PLAY STOP` stops whatever is playing. It asks the kernel which process holds the sound output rather than remembering what it started, so it also stops a player left running by a program that was stopped with Ctrl-C — which is the situation it is most often wanted for. Stopping when nothing is playing is not an error, as in MMBasic.

There is one sound output, so there is one player:

```
Error: sound output in use
```

is what `PLAY MP3` gives if something is already playing. Use `PLAY STOP` first. (This is MMBasic's "Sound output in use for . . . " under a shorter name.) The rule is enforced by the kernel rather than by convention: two decoders writing into the same buffer interleave their samples, and it sounds like it.

Files play at their own sample rate — 8 kHz to 48 kHz, mono or stereo, and a mono file costs nothing extra because the driver duplicates the channels itself. Copy MP3s onto the FAT partition from a PC and bring them over with `fat get`, as with any other file.

`PLAY` and BBC BASIC's `SOUND` share one piece of hardware and are mutually exclusive, exactly as they are in MMBasic. Starting a track silences the synthesiser; stopping it hands the hardware back.

6.15 Pins: SETPIN and PIN

```
SETPIN 2, DOUT
SETPIN 3, DIN
PIN(2) = 1
IF PIN(3) = 1 THEN PRINT "high"

SETPIN 41, AIN                  ' analogue, in volts
PRINT PIN(41)                   '   1.677936508
SETPIN 41, ARAW                 ' analogue, raw
PRINT PIN(41)                   '   2090
```

All **forty-eight** GPIOs of the RP2350B are available — the wider part is why the PC3 can put the real-time clock's alarm on GP32, which a 28-pin RP2040 could not reach.

`n` is the **GPIO number**, not MMBasic's connector-pin numbering. The GPIO number is what the schematic, the kernel, and every other tool on this machine use, and inventing a second numbering for one statement would cause more confusion than the incompatibility does.

`SETPIN` takes:

mode	what the pin becomes
`DIN`	a digital input
`DOUT`	a digital output, driven by `PIN(n) = v`
`AIN`	an analogue input read as **volts**
`ARAW`	an analogue input read as the **raw 0–4095 count**
`INTH`	a digital input that calls a SUB on a **low-to-high** edge
`INTL`	... on a **high-to-low** edge
`INTB`	... on **either** edge
`PWM`	a PWM output, driven by the `PWM` statement
`OFF`	not configured

MMBasic's remaining modes — frequency and counting — are not translated, and are reported by name.

Input modes take MMBasic's optional pull:

```
SETPIN 2, DIN, PULLUP
SETPIN 3, DIN, PULLDOWN
SETPIN 35, INTL, Pressed, PULLUP      ' after the handler
```

Absent means neither, which is MMBasic's default — a floating input then reads whatever the wire is doing, and with nothing connected that is noise. A switch to ground wants `PULLUP`.

Hysteresis is always on for inputs. It is not an option in MMBasic either: every digital input it configures gets the Schmitt trigger, and so does every one here. On a board with header pins and flying leads a slow or noisy edge is the normal case.

`AIN` and `ARAW` need an ADC pin: on the RP2350B channel n is GP40+n, so **GP40–GP46** on the I/O header. Anything else gives `Pin cannot do that`.

The two analogue modes differ only in what `PIN()` gives back. `ARAW` is a single conversion, which is what you want when you are going to do your own filtering or you care about speed. `AIN` is MMBasic's filter, step for step: ten readings, sorted, the top two and bottom two thrown away, and the remaining six averaged and scaled by 3.3 V. The point of it is the *discard* — one wild sample is dropped rather than smeared across the answer, which plain averaging would do. (MMBasic's `OPTION VCC` is not supported, so the scale is always 3.3 V.)

`PIN()` returns a **float in every mode**, where MMBasic returns an integer for digital pins and `ARAW`. Nothing observable changes — `PRINT PIN(2)` still prints `1`, and comparisons and array indices behave the same — because a double holds 0, 1 and every 12-bit count exactly. The reason is that translated C has to know the type when it is generated, and nothing at that point knows what mode a pin will be in.

Pins are now owned. `SETPIN` claims the pin from the kernel, and claiming one that another program holds gives `Pin cannot do that` rather than quietly fighting over it. The claim is released — and the pin **reset to an input** — when your program ends, however it ends, including being killed or crashing. So a program that dies driving a relay does not leave it driven.

Only the I/O header can be claimed: **GP0–GP7, GP26 and GP34–GP46**, plus **GP32**. The rest belong to the board (display, SD card, PSRAM, sound, console, the DS3231's I2C), and asking for one gives `Pin cannot do that` instead of taking a pin the audio hardware is driving. Appendix A lists what the board uses. A pin number outside 0–47 gives `Invalid pin`.

GP32 is not on the header — it is the **DS3231's alarm output**, and an alarm nothing can read is not an alarm. It is open-drain and active low, so with a pull-up it reads 1 until the alarm asserts:

```
SETPIN 32, DIN, PULLUP
PRINT PIN(32)                      ' 1 = not asserted

SETPIN 32, INTL, WakeUp, PULLUP    ' fires when it does assert
```

6.15.1 Arming the alarm

MMBasic has no alarm command — a program writes the chip's registers, and so does this:

```
SUB Alarm
  hits = hits + 1
  RTC SETREG 15, 0             ' clear the flag, or it never fires again
END SUB

RTC SETREG 7, 128              ' alarm 1 masks: match nothing = every second
RTC SETREG 8, 128
RTC SETREG 9, 128
RTC SETREG 10, 128
RTC SETREG 15, 0               ' clear any stale flag
RTC SETREG 14, 5               ' INTCN | A1IE - drive INT, enable alarm 1

SETPIN 32, INTL, Alarm, PULLUP
```

`RTC GETREG reg, var` and `RTC SETREG reg, value` reach any of the DS3231's registers, which is MMBasic's own interface (it has `GETREG` and `SETREG` and no alarm command). Register 7–10 are alarm 1, 0x0E is control, 0x0F is status. The chip's data sheet is the reference.

Your handler must clear the alarm flag. The INT line stays low until it is, so an alarm that does not clear it fires once and then looks broken.

Two things this cannot do. It will not stop the oscillator — a write to the control register comes back with `EOSC` masked out, because on a battery-backed part a stopped clock outlives the power cycle and the machine boots not knowing the time. And it is not `/dev/i2c`: that driver refuses address 0x68 outright, so this goes through the kernel's own path to the chip, sharing the retry and recovery the system clock already relies on.

On a **Pico Computer 2** GP32 is the SD card's MISO instead, so the same kernel refuses to hand it out there — the board tells itself apart at boot. This is the one pin whose availability depends on which machine you are running on.

`PIN(n) = v` on a pin that is not a `DOUT` gives `Pin is not an output`, and reading a pin that has never been `SETPIN`ed gives `Pin is not an input` — both as MMBasic does.

Pin work no longer costs a system call. `SETPIN` makes one, once, to claim; after that `PIN(n)` and `PIN(n) = v` are single register accesses, about ten nanoseconds against the 1.5 µs an ioctl costs. That is what makes bit-banging a protocol in BASIC possible at all.

6.16 Pin interrupts

```
SUB Pressed
  PRINT "GP35 went ";PIN(35)
END SUB

SETPIN 35, INTB, Pressed         ' either edge
DO
  ' ... ordinary work; the handler runs between statements
LOOP
```

The handler is an ordinary `SUB` taking no parameters, and it ends with `END SUB`. **There is no `IRETURN` to write** — that is MMBasic's behaviour too, not a simplification: for a SUB target MMBasic builds an `IRETURN` of its own and uses it as the return address of the `GOSUB` it fakes, so `END SUB` performs the interrupt return. Written `IRETURN` only ever existed for targets given as a label or a line number, and those are not translated: compiled code cannot jump into the middle of a function, so a label or line number is refused with a clear message rather than half-working.

These are not hardware interrupts, and they are not in MMBasic either. The whole facility is a poll. MMBasic's interpreter checks after every statement, and pin "interrupts" there are a comparison of the pin's level against its level at the previous check — there is no GPIO IRQ anywhere in it. This does the same thing at the same place, so the behaviour you get is the behaviour a PicoMite gives:

- **Latency is one statement**, and a statement is atomic. Nothing ever runs half a statement of your program.
- **A pulse shorter than a statement is missed.** That is MMBasic's documented limitation, replicated rather than "fixed".
- **Interrupts never nest.** A handler's own statements are polled, but the poll does nothing while a handler is running.
- **One handler runs per statement boundary.** If two pins change at once, the second fires at the next statement.

- `MM.ERRNO` and `MM.ERRMSG$` are **saved, cleared and restored** around a handler, so a handler starts clean and cannot leave an error behind for the interrupted code to trip over.

Up to ten pins may have interrupts, which is MMBasic's own limit. The level a handler reads with `PIN(n)` is the level *now*, not the level that triggered it — with a mechanical switch the two often differ, because contacts bounce faster than the handler starts. MMBasic reads it the same way.

A program that arms no interrupt pays nothing at all: the per-statement poll is emitted only for programs that use the feature, exactly as the `ON ERROR` checks are.

6.17 PWM

```
SETPIN 0, PWM                ' GP0 is slice 0, channel A
PWM 0, 1000, 25              ' slice 0: 1 kHz, 25% on channel A
PWM 0, 1000, 25, 75          ' ...and 75% on channel B
PWM 0, 1000, -25             ' negative duty = inverted output
PWM 0, OFF
```

`SETPIN pin, PWM` attaches a pin to its PWM output; the `PWM` statement then drives the **slice**, not the pin. One slice has two channels and drives two pins, which is why the two commands are separate — the same split MMBasic uses.

Which slice a pin belongs to is arithmetic: pins below GP32 use slice `(pin ÷ 2) MOD 8`, and GP32 upward use `8 + ((pin ÷ 2) MOD 4)`. Even pins are channel A, odd pins channel B. **Twelve slices cover forty-eight pins, so pins alias**: GP34 and GP42 are the same slice and channel, and setting one sets the other. `SETPIN pin, PWM` claims the slice as well as the pin, so a second program asking for a pin that lands on a slice you hold is told `Pin cannot do that` rather than quietly changing your frequency.

Frequency is in Hz and duty in percent, and the range is wide: 100 kHz and 50 Hz both work. Below about 5.7 kHz the 16-bit counter cannot hold a whole period at 375 MHz, so the clock divider takes up the slack — that happens automatically, and the only sign of it is that very low frequencies have coarser duty resolution. Asking for something the divider cannot reach either gives `Invalid frequency`.

A **negative duty** inverts that channel's output: `-25` is 25% inverted, so the pin reads high 75% of the time. It is the channel's polarity bit, not a recomputed duty.

`PIN()` will not read a PWM pin — it is an output, and MMBasic refuses it too.

Giving only one duty leaves the other channel alone. That is not a detail: two outputs share a slice, so `PWM 0, 1000, 25` must not stop whatever is running on channel B.

6.18 SERVO

```
SETPIN 0, PWM
SERVO 0, 50                  ' centre
SERVO 0, 0, 100              ' two servos on one slice, opposite ends
SERVO 0, OFF
```

A servo is PWM at a fixed 50 Hz frame with the position expressed as a pulse width, so `SERVO` is `PWM` with one line of arithmetic in front of it — MMBasic's:

position	pulse	
0	1.0 ms	one end
50	1.5 ms	centre
100	2.0 ms	the other end
-20	0.8 ms	over-travel
120	2.2 ms	over-travel

The range is -20 to 120, which is the over-travel most servos will accept; outside it you get `Invalid servo position`. Everything else is the `PWM` statement, including the rule that omitting the second position leaves that channel alone — which is what lets two servos share a slice.

Scope-verified on GP0 at all five positions.

6.19 `SETTICK` — periodic timers

```
SUB Every100
  INC count
END SUB

SETTICK 100, Every100          ' every 100 ms
SETTICK 100, Every100, 2       ' timer 2 of 4
SETTICK PAUSE, 2               ' freeze it where it stands
SETTICK RESUME, 2
SETTICK 0, 0, 2                ' off
```

Four timers, numbered 1 to 4, period in milliseconds — MMBasic's `NBRSETTICKS`. The number is optional and defaults to 1. The handler is a parameterless SUB, as for pin interrupts, and everything said above about them applies here too: it runs between statements, it does not nest, and one interrupt is dispatched per statement boundary. A pin edge and a tick falling due at the same moment dispatch the pin first, which is MMBasic's scan order.

Missed ticks are dropped, not queued. If a handler takes longer than its own period, the timer catches up by whole periods and fires once — so a slow handler cannot spiral into a backlog it never clears. The phase is kept, so the cadence does not drift.

Measured on the board: twenty ticks of 100 ms took **1999.98 ms**, and a 10 ms timer running beside a 50 ms one fired exactly 100 times in the second the slow one took to reach twenty.

6.20 `ON KEY` — the console

```
SUB AnyKey
  k$ = INKEY$                  ' the key is still there for you
END SUB

SUB QuitKey                    ' fires on "q" only, and eats it
  finished = 1
END SUB

ON KEY AnyKey                  ' any key
```

```
ON KEY 113, QuitKey           ' just this one
ON KEY 113, 0                 ' that one off
ON KEY 0                      ' the any-key one off
```

Two forms, and **what happens to the key is the difference between them** — this is MMBasic's design, not an accident of ours:

- `ON KEY handler` fires while a key is *waiting*, and the key stays waiting. Your handler reads it with `INKEY$`. If it doesn't, the handler fires again at the next statement, because the key is still there.
- `ON KEY code, handler` fires only on that character code, and **consumes** it. It never reaches `INKEY$`, so a key reserved for a hot-key handler cannot also turn up in the program's ordinary input.

The specific form is checked first, so with both armed the reserved key goes to its own handler and everything else goes to the general one.

Two things to know before relying on it:

The console is checked at most every 5 ms, not after every statement. Looking is a system call, and a poll site runs after every statement; checking each time would cost far more than most statements do. Five milliseconds is the kernel's own tick and far below what anyone can type or perceive, but it is a divergence from MMBasic, which looks every time.

A program reading keys holds the terminal, and that has visible consequences worth knowing.

The console has a line editor in front of it — the thing that gives you history and cursor keys at the shell prompt. It takes every keystroke while the terminal is in cooked mode with echo, and hands over a finished line only when you press Enter. A program that wants *keystrokes* has to stand it down, so the first `INKEY$` or armed `ON KEY` puts the terminal into raw mode and **keeps it there** for the rest of the program.

While it is held:

- **keys are not echoed** — nothing appears on screen unless your program prints it. This is MMBasic's behaviour for `INKEY$` too.
- **there is no line editing** — no backspace, no history, no cursor keys. Each keystroke goes straight to the program.
- **`INPUT` still works normally.** It gives the terminal back for the duration of the question and takes it again afterwards, so a program can mix `INKEY$` with `INPUT` and the typed answer is edited and echoed as usual.
- the terminal is **restored when the program ends**, however it ends.

This is what BBC BASIC and the editors on this machine already do.

6.21 Timers and the deliberate divergence

The 100 ms figure above is the one **deliberate divergence** from MMBasic, and it is in your favour. MMBasic counts milliseconds in an interrupt and fires when the count is *greater than* the period, so a `SETTICK 100` there actually runs at 101 ms and a 16 ms game timer at 17 ms — about 6% slow. This keeps a microsecond deadline instead of a millisecond counter and fires at the period you asked for. Nothing else about the behaviour differs; if you are porting a program whose timing was tuned against a PicoMite, it will run very slightly faster here.

Every read and write is a call into the kernel — comparable to drawing one pixel, which is over a microsecond. That is ample for a switch or an LED and nowhere near enough to bit-bang a protocol; write that in C, or use the hardware that already exists for it.

6.22 Practical notes

- One `.bas` file per program, because `cc` compiles one file per program. `Sub`s and `Function`s all live in that file.
- The C file is a normal file — you can keep it, edit it, or hand it to `cc` on another day.
- `mmbc` and `cc` each need a little room. On a machine with other things running, a large program is happier if you are not also holding a big file open in an editor.
- Programs built this way are the same objects `cc` produces from hand written C, so `bcdump` disassembles them and `BCRUN_BYTECODE=1` forces interpretation for comparison.

6.23 A worked example: a barometric station

Everything in this chapter, on one screen. The program is `qnh.bas` in `/root/cc`; what follows is how it is built rather than a listing of it.

It reads a BMP180 barometer, a potentiometer and the clock, and puts the results on an ILI9341 panel that the firmware has never heard of — including the text, drawn from the built-in fonts:

```
GP2  SCLK    GP3  MOSI    GP4  MISO     the panel
GP5  DC      GP6  RESET   GP7  CS       its control lines
GP0  LED                                backlight, PWM slice 0
GP38 SDA     GP39 SCL                   I2C2, the BMP180 at &H77
GP41                                    the potentiometer wiper
```

Build and run it the usual way:

```
# mmbc qnh.bas
# cc qnh.c
# ./qnh.bc
```

6.23.1 What it is for

A barometer reads the pressure where it is standing. Weather reports quote pressure **reduced to sea level**, because otherwise a reading would say more about your altitude than about the weather — the difference is about 12 hPa per hundred metres, which is the gap between a settled day and a gale.

The reduction needs to know how high you are, and that is what the potentiometer sets. The formula is the one in the BMP180's own datasheet, inverted:

```
qnh = pHPa / (1 - alt / 44330.0) ^ 5.255
```

The result is QNH: what an altimeter should be set to so it reads height above sea level.

6.23.2 Reading the sensor

The BMP180 arithmetic is Bosch's, out of the datasheet, and the program uses MMBasic's own BMP180 listing unchanged apart from the bus — the part is on the I/O header rather than the QWIIC socket, so `SETPIN` names the pins and `I2C2` opens the second controller:

```
SETPIN 38, 39, I2C2
I2C2 OPEN 400, 1000
I2C2 WRITE &H77, 1, 1, &HAA        ' the calibration block
I2C2 READ  &H77, 0, 22, i2cin$
ac1 = STR2BIN(int16, MID$(i2cin$, 1, 2), big)
```

Twenty-two bytes of calibration come back as a string and `STR2BIN` picks signed and unsigned, big-endian words out of it.

6.23.3 Reading the potentiometer

One line of setup, because `AIN` is MMBasic's:

```
SETPIN 41, AIN
alt = INT(PIN(41) / 3.3 * 1000 + 0.5)
```

`PIN` on an `AIN` pin gives volts: ten readings, sorted, the top two and bottom two discarded and the remaining six averaged, scaled 3.3 V over 4095 — MMBasic's filter, constant for constant. On the RP2350B channel n is GP40 + n, so GP41 is channel 1.

One ADC count is about a quarter of a metre over a 1000 m range, so the program ignores movements below two metres. A display that flickers between 141 and 142 is worse than one that is a metre out.

6.23.4 Drawing text on a panel the firmware does not know

This is the part that needs `MM.INFO(FONT ADDRESS)`. The fonts are cached once — address, cell and range, every value read out of the font's own header:

```
FOR f = 1 TO 9
  a = MM.INFO(FONT ADDRESS f)
  fadr(f) = a
  IF a <> 0 THEN
    fwid(f) = PEEK(BYTE a)
    fhgt(f) = PEEK(BYTE a + 1)
    ffst(f) = PEEK(BYTE a + 2)
    fcnt(f) = PEEK(BYTE a + 3)
  ENDIF
NEXT f
```

Drawing one character is then: find its glyph, set the panel's window to exactly one cell, and write the rows. The window is set once and the panel advances by itself, so a glyph costs one address round trip and one write per row rather than one per pixel:

```
g = fadr(f) + 4 + (c - ffst(f)) * w * h \ 8
setwin(x, y, x + w - 1, y + h - 1)
FOR row = 0 TO h - 1
  px$ = ""
  FOR col = 0 TO w - 1
    bidx = row * w + col
    bval = PEEK(BYTE g + bidx \ 8)
    IF (bval >> (7 - (bidx AND 7))) AND 1 THEN
      px$ = px$ + ink$
```

```
    ELSE
      px$ = px$ + paper$
    ENDIF
  NEXT col
  SPI WRITE w * 2, px$
NEXT row
```

`ink$` and `paper$` are two-byte RGB565 pixels built once per call, so the inner loop appends rather than converting.

A row is `width × 2` bytes, which is why this works within the 255-byte string limit: 32 bytes for the 16×24 font, 48 for the 24×32. A whole glyph would not fit — the 16×24 one is 768 bytes — but a row always does.

6.23.5 Two traps this example exists to show

The panel does not refuse a window wider than itself. It clamps the window and still accepts every pixel you write into it, so the surplus wraps onto the next row and the character comes out as noise. Fifteen characters of the 16-pixel font is 240 pixels — the whole panel — so a title starting at x=8 loses its last letter into the row below. The program therefore drops a glyph that would overhang rather than drawing it:

```
gx = x + (i - 1) * w
IF gx + w > SW THEN EXIT FOR
```

Silently short is a layout mistake you can see. Silently wrapped looks like a bug in the font reader, and will cost you an evening.

Mid grey is not visible. RGB565 `&H8410` is a perfectly reasonable grey and reads as almost nothing on this panel at 70 % backlight. `&HBDF7` — about three quarters — is legible. Colours that look fine in a table do not always survive a real display, and there is no substitute for looking at it.

6.23.6 What the translation looks like

`MM.INFO(FONT ADDRESS f)` becomes a runtime call and `PEEK` becomes a load, which is the whole point of the exercise — the address is not marshalled anywhere, it is used:

```
v_a = mm_fontaddr(v_f);
H->v_fadr[(int)(v_f)] = v_a;
if (((v_a) != (0LL))) {
    H->v_fwid[(int)(v_f)] = mmpk_byte(v_a);
    H->v_fhgt[(int)(v_f)] = mmpk_byte(((v_a) + (1LL)));
    H->v_ffst[(int)(v_f)] = mmpk_byte(((v_a) + (2LL)));
    H->v_fcnt[(int)(v_f)] = mmpk_byte(((v_a) + (3LL)));
}
```

(`H->` is where arrays and strings live — one allocation the program owns, rather than globals.)

and `mmpk_byte` is one line in `mmb_peek.h`:

```
MMG_FN MMINTEGER mmpk_byte(MMINTEGER addr)
{
```

```
    return (MMINTEGER)*MMPK_PTR(unsigned char, addr);
}
```

Reading font data compiles to a `ldrb`. There is no MMU to go through, no copy, and no system call — the flash the kernel keeps its fonts in is simply memory that this program can read.

The whole program is 15 KB of BASIC and compiles to about 63 KB of bytecode.

7 `mmedit`: MMBasic's editor

MMBasic's full-screen editor is ported and runs as an ordinary Fuzix program, so BASIC is written, translated, compiled and run without leaving the machine:

```
# mmedit prog.bas
```

It is the editor from the firmware, with the same keys:

Key	
`ESC`	leave the editor
`F1`	save
`F2`	save and exit (so a wrapper can run the program)
`F3` / `F6`	find / find again
`F4`	mark — then the cursor keys extend the selection
`F5`	paste
`F7` / `F8`	replace / replace again
`F9` / `F10`	read a file in / write the selection out
`F12` or `Ctrl-A`	beautify: re-indent and case the keywords

In mark mode the legend changes: `DEL` deletes, `F4` cuts, `F5` copies, `F10` exports the selection to a file. The status line shows line, column and INS/OVR, and the function key legend shortens itself to fit the terminal width.

The colour coding answers "will this compile?" A keyword `mmbc` can translate is cyan; one only the interpreter knows is blue. That second colour is the useful one here, because a program is compiled rather than run as you type it, and it is worth seeing `MM.WATCHDOG` or `BLIT` in blue before you build rather than after. The lists come from the translator's own tables, so they cannot drift from what it does.

The editor works over the serial port as well as on the HDMI console — it drives a VT100, and the console emits VT100 function key sequences. Files with CRLF line endings are accepted; the CRs are stripped on load, which matters because most files arrive from a PC.

It takes the screen back if a program left it in a graphics mode. The editor draws on the text console, which is what `MODE 1` selects. A program that finished in `MODE 2` leaves the screen 320×240 in sixteen colours, and the console is then not what the monitor is showing — so the editor would be painting where nothing can be seen. `mmedit` switches to `MODE 1` on the way in and puts the old mode back on the way out, including when it is killed, so a program's screen survives a trip through the editor and comes back as it was.

7.1 The other editor: `vi`

`mmedit` is for BASIC. For small text files — a shell script, `/etc/motd`, a short C file — there is a `vi`:

```
# vi hello.c
```

It is Levee, David Parsons' small vi clone, with modal editing, the `:` commands and the usual movement keys. It has been on the card since the beginning under its own name, `levee`, which still works: `vi` and `levee` are two names for one file.

Its edit buffer is 64 KB, which is enough for any source file you are likely to write on the machine. Levee's own default is 4 KB — it was written for 8-bit micros, where that was a fair share of the machine — and this port raises it, since a process here may have most of a 340 KB pool to itself.

Open something larger than the buffer and Levee says `[overflow]` on the status line and holds the file **read-only**: you can look and move around, but `:w` answers "File is readonly" rather than writing a truncated file back. That is the safe way round, and worth knowing as the signal that a file is too big rather than that something is wrong.

One difference from a full `vi` to watch: `cw` on the last word of a line joins the following line onto it.

8 The FAT partition and the `fat` command

Partition 1 of the SD card is ordinary FAT, readable and writable by any PC — this is how files travel to Fuzix, because nothing else can mount the Unix partition. Format it once in Windows (FAT or FAT32, **not exFAT**), then simply copy files onto it.

On the Fuzix side, the `fat` command reads it (long filenames and subdirectories included):

```
# fat info
/dev/hdb1: FAT16, 32695 clusters of 2048 bytes (62 MB)
# fat ls
    4523  My Program.bas
   <dir>  BASIC
# fat ls basic
# fat get "my program.bas" /root/prog.bas
# fat get demo.bin                    (lowercased name in current dir)
```

`fat` is read-only by design — the desktop does the writing. To move a file *out* of Fuzix, use the serial port, or write it with `doswrite` (the venerable Minix FAT12/16 tool, also included) if the partition is FAT16.

9 Moving files over the serial port

The FAT partition carries files *in*, but `fat` is read-only, so it cannot carry anything back *out*. The serial console can do both, in either direction, with nothing on the board but `uue` and `uud` — and it is the only route that needs no card swapping.

The idea is older than the machine: `uuencode` turns any file into printable text, and printable text survives a terminal. Both tools are in `/bin`:

```
# uue FILE            writes FILE.uue - the encoded text
# uud FILE.uue        decodes it back, recreating the original name
```

9.1 Sending a file to the board

On the PC, encode the file, then have the board read the text into a file and decode it:

```
# cat > prog.uue          (now paste or send the encoded text)
^D
# uud prog.uue
# ls -l prog.bas
```

The one thing that will bite you is flow control. The terminal input queue is 132 bytes. Sending encoded text as fast as the port will take it overruns that queue and loses characters silently — a 62-character line sent every 25 ms still loses roughly 60% of the text, and the damage only shows up as a corrupt file at the end. Two ways to avoid it:

- **Let the terminal pace it.** Any sender with a per-line delay *and* a wait for the echo will do. The delay alone is not enough.
- **Use the supplied script.** `devtools/uusend.py` in the source tree does exactly this: it types the text one line at a time and waits for each line to echo before sending the next, so at most one line is ever in flight. It then runs `uud` for you.

```
python uusend.py local.bas prog.bas --port=COM11
```

Note that `uusend.py` takes a **bare** remote filename — it drives one `ucp`-style session with `cd` and plain names, so `mv` the file afterwards if it belongs somewhere else.

9.2 Fetching a file from the board

The other direction is easier, because the board is the slow end:

```
# uue report.txt
# cat report.uue
```

Capture the session to a file in your terminal program, cut away anything before `begin` and after `end`, and decode it on the PC.

9.3 Programs to use

Linux (and macOS, and WSL):

- `uuencode` / `uudecode` come from the **sharutils** package (`apt install sharutils`). macOS has `uuencode` built in.

- For the terminal, **picocom** (simplest), **minicom** (its `Ctrl-A S` menu includes an `ascii-xfer` that paces lines), or **screen /dev/ttyUSB0 115200**.
- If sharutils is awkward, Python needs no install: `python3 -c "import binascii,sys; ..."` — or just use `devtools/uusend.py`, which does the encoding itself.

Windows:

- **Tera Term** is the pick — free, and its *File → Send file* has a per-line delay setting under *Setup → Serial port* (set a transmit delay of a few ms per character or 30–50 ms per line). This is the easiest reliable route without scripting.
- **PuTTY** works for capture (*Session → Logging*) but has no paced file send; pasting a large file into it will lose characters.
- **RealTerm** can send a file with a configurable delay, and is good at capturing binary.
- For the encode/decode step, Windows has no built-in `uuencode`. Use **WSL** (then it is the Linux instructions), **Python** for Windows, or **UUDeview**, which still builds and runs.

Whatever you use, the check at the end is the same: `sum` the file on both machines and compare. On the board, `sum FILE`.

10 The filesystem and included software

10.1 Layout

```
/bin          core utilities (always available)
/usr/bin      larger tools, BBC BASIC, fat, picoctl
/usr/games    the games collection
/usr/lib/bbc  BBC BASIC library directory (@lib$)
/usr/lib/cc   the C compiler passes, and its headers in include/
/usr/man      manual pages (man <name>)
/dev          devices - see below
/etc          rc (boot script), inittab, passwd, termcap
/tmp          scratch space
/root         root's home directory
```

Key devices:

Device	What it is
`/dev/tty1`	The console (HDMI + USB keyboard + serial mirror)
`/dev/tty2`	The GP0/GP1 serial port
`/dev/hda`	On-board NAND flash filesystem
`/dev/hdb1`–`hdb3`	SD card partitions (root is `hdb2`)
`/dev/rtc`	The DS3231 clock (`setdate` reads and sets it)
`/dev/sys`	Platform control (graphics, sound, ADVAL ioctls)

`/etc/rc` runs at boot: filesystem check, clock from the DS3231, keyboard layout. Edit it with any of the editors below.

There is no swap device to enable. The PSRAM is not a block device any more: the kernel takes an allocation the size of the process when it needs to swap one out, and programs take their heap from the same place. `free` reports both.

10.2 Applications

Editors: `mmedit` (MMBasic's own full-screen editor — see its chapter), `vi` (levee), `ue` (a WordStar-diamond micro-Emacs: Ctrl-W write, Ctrl-Q quit), `ed`, `fleamacs`.

Shell & core: Bourne `sh` (plus `fsh`), and the classic set — `ls ll cp mv ln rm mkdir rmdir cat more less head tail grep fgrep sed tr cut sort uniq wc find xargs diff diff3 cmp comm join split rev tar dd df du free ps kill killall uptime date cal banner echo sleep tee touch which who su passwd stty mount umount sync fsck mkfs fdisk chmod chown chgrp od hd factor seq units dc expr m4 make cron at mail write wall` (and more — see `ls /bin /usr/bin`).

Machine tools: `picoctl` (keyboard layout, reboot to the flasher), `picogpio/gpiotool`, `gfxtest` (display test card), `setdate` (DS3231), `flashrom`, `setboot`, `dosread/doswrite` (FAT12/16 floppy-era transfers), `fat`, `uue/uud` (serial file transfer — see that chapter).

Images: `saveimage` writes any part of the screen as a 24-bit BMP, `loadimage` puts one back:

```
# saveimage shot.bmp                    the whole screen
# saveimage part.bmp 160 120 320 240    x y w h
# loadimage shot.bmp                    at 0,0
# loadimage tiger.bmp 0 0 4             with dither mode 4
```

`loadimage` is MMBasic's BMP decoder, so it reads 1/4/8/16/24/32-bit files, `BI_BITFIELDS`, and RLE4/RLE8, and dithers only when asked. These are the programs `SAVE IMAGE` and `LOAD IMAGE` run, and they are just as useful from the shell.

Sound: `playmp3` plays an MP3 file through the audio DAC:

```
# playmp3 track.mp3           volume 80 by default
# playmp3 track.mp3 40        0 to 100
# playmp3 track.mp3 &         in the background, and carry on working
```

It decodes at about six times real time and takes almost nothing from whatever else is running. This is the program `PLAY MP3` runs, and one plays at a time — a second is refused by name and pid rather than allowed to talk over the first. Stop one with `kill -2` (or, from BASIC, `PLAY STOP`).

Languages: `bbcbasic`, `cc` (the on-board C compiler — see its own chapter, with `cpp`, `bcrun` and `bcdump`), `fforth` (a complete ANS Forth), the `as09`/`ld09` assembler pair, and `dc`.

Games (`/usr/games`): the original Colossal Cave `advent`, the complete Scott Adams `adv01`–`adv14` and Mysterious Adventures `myst01`–`myst11` collections, Infocom Z-machine interpreters `z1`–`z8` and `l9x` (Level 9), `startrek`, `hamurabi`, `backgammon`, `invaders`, `2048`, `moo`, `ttt`, `fish`, `arithmetic`, `fortune`, `cowsay`, `wump`.

11 Appendix A: pin usage

GPIO	Function
GP0 / GP1	Serial port `/dev/tty2` (TX / RX)
GP8 / GP9	System console UART (USB-C CH340)
GP10/11/22	Audio I2S: BCLK / LRCLK / DATA (PCM5102 DAC)
GP12–GP19	HDMI (HSTX)
GP20 / GP21	I2C0: DS3231 RTC and QWIIC connector (SDA / SCL)
GP27	DS3231 32 kHz — board identification (PC3 only)
GP28/30/31/33	SD card, Pico Computer 3 (MISO/SCK/MOSI/CS, SPI1)
GP29/30/31/32	SD card, Pico Computer 2 (CS/SCK/MOSI/MISO, bit-banged)
GP32	DS3231 alarm interrupt (PC3; unused by Fuzix)
GP34–GP37	Joystick switches — `ADVAL(0)` bits 0–3
GP40	Analogue input (reserved; not read by ADVAL)
GP41–GP44	Analogue inputs — `ADVAL(1)`–`ADVAL(4)`
GP45/GP46	Analogue-capable, free
GP23/24/25/29	CYW43 wireless (PC3; unused by Fuzix)
GP47	PSRAM chip select

All spare header pins remain ordinary 3.3 V GPIO. Do not apply 5 V to any GPIO.

12 Appendix B: boot options

The kernel command line (held by the bootloader) accepts:

- `hda` / `hdb2` … — root device override
- `kbd=us|uk|de|fr|es|be` — early keyboard layout override (the layout in `/etc/rc` then applies for the session)

The build, source and development notes live in the `pc3` branch of `github.com/UKTailwind/FUZIX` under `Kernel/platform/platform-rpipico/` — see `PC3-DEVNOTES.md` for the engineering history and `PC3-GFX-DESIGN.md` for the display design.

13 Appendix C: MMBasic coverage

This is what `mmbc` translates today. It is generated from the translator's own tables (`fcc/coverage.py` in the mmb2c repository), so it says what the program does rather than what anyone remembers it doing.

Coverage grows with each release, and it will never be complete. MMBasic is a large language whose statements reach deep into one particular firmware, and a translator that emits portable C cannot follow all of it. Anything not listed here is reported by name, with its line number, and the translation continues - so you find out at translate time, not at run time.

13.1 Statements

`?`	`ARC`	`ARRAY`	`BOX`
`CALL`	`CASE`	`CAT`	`CHDIR`
`CIRCLE`	`CLEAR`	`CLOSE`	`CLS`
`COLOR`	`COLOUR`	`CONST`	`CONTINUE`
`COPY`	`DATA`	`DATE$`	`DIM`
`DO`	`ELSE`	`ELSEIF`	`END`
`ENDIF`	`ERASE`	`ERROR`	`EXIT`
`FILES`	`FONT`	`FOR`	`FRAMEBUFFER`
`FUNCTION`	`GOSUB`	`GOTO`	`I2C2`
`IF`	`INC`	`INPUT`	`KILL`
`LET`	`LINE`	`LOAD`	`LOCAL`
`LONGSTRING`	`LOOP`	`MAP`	`MATH`
`MKDIR`	`MODE`	`NEXT`	`ON`
`OPEN`	`OPTION`	`PAUSE`	`PIN`
`PIXEL`	`PLAY`	`PRINT`	`PWM`
`RANDOMIZE`	`RBOX`	`READ`	`RENAME`
`RESTORE`	`RETURN`	`RMDIR`	`RTC`
`SAVE`	`SEEK`	`SELECT`	`SERVO`
`SETPIN`	`SETTICK`	`SORT`	`SPI`
`STATIC`	`STRUCT`	`SUB`	`SYSTEM`
`TEXT`	`TIME$`	`TIMER`	`TRIANGLE`
`TYPE`	`WEND`	`WHILE`	

Assignment needs no keyword (`LET` is accepted). Statement separators, line numbers and labels, `REM` and `'` comments all work as expected.

13.2 Functions

`ABS`	`ACOS`	`ASC`	`ASIN`
`ATAN2`	`ATN`	`BIN$`	`BIN2STR$`
`BIT`	`BOUND`	`BYTE`	`CHOICE`
`CHR$`	`CINT`	`COS`	`CWD$`
`DATE$`	`DATETIME$`	`DAY$`	`DEG`
`DIR$`	`EOF`	`EPOCH`	`EXP`

FIELD$	FIX	FORMAT$	HEX$
INKEY$	INPUT$	INSTR	INT
LCASE$	LCOMPARE	LEFT$	LEN
LGETBYTE	LGETSTR$	LINPUT	LINSTR
LLEN	LOC	LOF	LOG
LTRIM$	MAP	MATH	MAX
MID$	MIN	MM.CMDLINE$	MM.DEVICE$
MM.ERRMSG$	MM.ERRNO	MM.HRES	MM.INFO
MM.SPISPEED	MM.VER	MM.VRES	OCT$
PEEK	PI	PIN	PIXEL
RAD	RGB	RIGHT$	RND
RTRIM$	SGN	SIN	SPACE$
SPI	SQR	STR$	STR2BIN
STRING$	STRUCT	TAB	TAN
TIME$	TIMER	TRIM$	UCASE$
VAL			

13.3 MATH() sub-functions

Scalar: `ATAN3`, `COSH`, `LOG10`, `SINH`, `TANH`

Whole-array (one number out of an array): `MAX`, `MEAN`, `MEDIAN`, `MIN`, `SD`, `SUM`

13.4 MATH sub-commands

`MATH` is also a statement, and that is a different and much longer list in the interpreter. Four of it are translated — the ones that walk an array element by element, which is what most programs use it for:

`MATH SET v, a()`	every element of `a()` becomes `v`
`MATH ADD a(), v, b()`	`b() = a() + v`, element by element
`MATH SCALE a(), v, b()`	`b() = a() * v`, element by element
`MATH RANDOMIZE [seed]`	seed the generator; no seed uses the clock

All four take integer, float or string arrays, except `SCALE`, which is numeric only — as it is there. `ARRAY` is accepted as a spelling of `MATH` for these.

The rest are not translated, and each says so by name rather than being mistaken for something else: the matrix operations (`M_MULT`, `M_INVERSE`, `M_TRANSPOSE`, `M_PRINT`), the vector ones (`V_MULT`, `V_CROSS`, `V_NORMALISE`, `V_ROTATE`, `V_PRINT`), the quaternions (`Q_CREATE`, `Q_EULER`, `Q_INVERT`, `Q_MULT`, `Q_ROTATE`, `Q_VECTOR`), the complex arithmetic (`C_ADD`, `C_SUB`, `C_MUL`, `C_DIV`, `C_AND`, `C_OR`, `C_XOR`), `FFT`, `WINDOW`, `SINC`, `INTERPOLATE`, `POWER`, `SHIFT`, `SLICE`, `INSERT`, `PID`, `SENSORFUSION` and `AES128`.

They are a coherent block of work rather than a scattering of gaps — most are pure arithmetic over arrays with no hardware in them, so they would go in as a header of static functions and cost nothing to a program that does not use them.

13.5 Types and structure

`INTEGER` (64-bit), `FLOAT` (double), `STRING`, and arrays of each, up to the dimensions MMBasic allows. `DIM`, `LOCAL`, `STATIC`, `CONST`, `OPTION BASE`, `SUB` and `FUNCTION` with by-reference arguments, and the usual control flow.

Structures (MMBasic's `TYPE ... END TYPE`, documented in full in the PicoMite structures manual) are translated with the firmware's byte layout reproduced exactly — `STRUCT(SIZEOF "t")` answers the same number here and on a PicoMite. Members may be `INTEGER`, `INT`, `FLOAT`, `STRING [LENGTH n]`, arrays of those, or an earlier TYPE; member access works to the full nesting depth (`data(2).items(1).values(4)` included), structure variables, arrays, `LOCAL`s and by-reference parameters all work, whole structures assign with `=`, and `STRUCT COPY`, `STRUCT CLEAR` and `STRUCT SWAP` are in. `STRUCT(SIZEOF/OFFSET/TYPE)` fold to constants when the names are literal strings.

Not translated (each says so rather than mistranslating): `STRUCT SORT/SAVE/LOAD/PRINT/EXTRACT/INSERT`, `STRUCT(FIND)`, a `FUNCTION` returning a structure, whole structure arrays as parameters, and initialisers on structure arrays. Assigning a whole structure into or out of a *nested* member is refused deliberately: the interpreter copies the outer type's size there and overruns memory, and a clean error beats reproducing that.

13.6 Errors, and surviving them

In a program that uses `ON ERROR`, every error it can hit is a check the runtime makes *before* it does the thing — the same order the interpreter uses. Nothing is left to the hardware to notice: dividing by zero as a float would otherwise answer `inf` rather than stopping, and this processor does not trap integer division by zero at all.

A program with no `ON ERROR` anywhere is compiled without the arithmetic checks — see "What `ON ERROR` costs" below. Its float divisions and `SQR/LOG/ASIN/ACOS` take what C gives: `inf` or `nan`, flowing onward through the sums. There is no sensible recovery from arithmetic like that — the program has a bug to fix either way — so the choice is between stopping with a message (a trapping program) and full speed (everything else). Integer division and `MOD` are the exception, checked in every program: C leaves integer division by zero undefined, and the silent zero this processor answers is not a number to limp onward with.

`ON ERROR SKIP [n]`, `ON ERROR IGNORE`, `ON ERROR CLEAR` and `ON ERROR ABORT` work as they do on a PicoMite, with `MM.ERRNO` and `MM.ERRMSG$` reporting what happened. A statement that fails while an error is being skipped is abandoned where it failed: the assignment does not happen, the rest of the `PRINT` does not print, and execution resumes at the next statement — in the `SUB` it happened in, if that is where it was. The count works as it does there too, `SKIP n` covering the next n statements.

Two details that surprise people, both matching the interpreter. A `PRINT` that fails part way through prints **nothing at all** — not even the items before the failure — because the line is built whole and the error discards it. And calling a `SUB` spends skip count: the `SUB` line and any `LOCAL` are statements too, so `ON ERROR SKIP 2` before a call does not reach the third statement inside it.

Two limits worth knowing. A statement that jumps away (`GOTO`, `EXIT`) does not count against `SKIP n`, so a count that spans one runs one statement further than it would on a PicoMite. And running out of memory ends the program whatever `ON ERROR` says — there is nothing sensible to carry on with, and a program limping along on a failed allocation would fail somewhere far less obvious.

`ON ERROR RESTART` reboots the machine on a PicoMite; a compiled program has no equivalent, so mmbc refuses it by name rather than guessing.

13.6.1 What ON ERROR costs

The checks are real work in real loops. With ON ERROR present, every float division whose divisor is not a nonzero literal becomes a runtime call that tests the divisor first, and SQR, LOG, ASIN and ACOS go through checked wrappers instead of straight to libm — that is what makes their errors trappable statements. Measured on the machine from a fresh boot: the grains benchmark, whose inner loop divides by a variable and takes a logarithm every pass, pays about 12%; the solar eclipse predictor, dominated by plain arithmetic, about 2%. The string checks (concatenation past 255 characters, ASC of an empty string) are part of string calls that happen anyway and stay on in every program; they cost a comparison, not a call.

ON ERROR earns its keep trapping things that genuinely can fail at run time — an I2C transfer that may not be acknowledged, a file that may not exist — not arithmetic. If a program traps errors out of habit, leaving ON ERROR out buys the checks back.

13.6.2 I2C2 — the second controller

I2C2 is MMBasic's second bus, and on this machine it is the one a program may have. The fixed bus (I2C, GP20/GP21) is the QWIIC socket *and* the DS3231 the system clock runs on, which the kernel polls from interrupt context, so it stays the kernel's.

```
SETPIN 38, 39, I2C2          ' GP38/GP39 or GP42/GP43 - see below
I2C2 OPEN 400, 1000          ' speed kHz, timeout ms
I2C2 WRITE &H77, 1, 1, &HD0  ' option 1 = HOLD: no STOP
I2C2 READ  &H77, 0, 1, r$    ' ...so this is a repeated START
I2C2 CLOSE
```

The pins are not free to choose: the RP2350 muxes I2C1's SDA only onto pins where `pin AND 3 = 2` and SCL only where `pin AND 3 = 3`, so on this board the pairs are **GP38/GP39** and **GP42/GP43**. Anything else is refused at OPEN rather than producing a bus that never answers.

WRITE takes a list of byte expressions, a whole numeric array written `a()`, or a string; READ takes a numeric array or a string variable. The string forms are what MMBasic's own sensor examples use, because STR2BIN() then lifts the 16- and 32-bit fields straight out of what was read.

Option bit 0 **holds** the bus: the transfer ends without a STOP so the next one is a repeated START on the same device. Most register-file devices do not need it — they keep their address pointer across a STOP — but the ones that do would otherwise read plausible nonsense.

Two differences from a PicoMite, both deliberate:

- `timeout` is milliseconds, 0 or 100 and up as MMBasic requires, but **0 means a five-second cap, not "no timeout"**. This kernel is non-preemptive: a transfer that waited for ever would not hang the program that asked for it, it would hang the machine, console and display included.
- A held bus that is never finished — the program errors, or is killed between the write and the read — is finished by the kernel, which clocks SCL until the device releases SDA and then issues a STOP. Nothing else on the board would free it.

The pins and the controller are claimed through the pin lock, so they come back if the program dies, and a second program asking for the bus gets a clear "already in use" rather than a collision.

13.6.3 SPI — the first controller

```
SETPIN 2, 3, 4, SPI          ' any order: the pin decides its own role
SPI OPEN 62500000, 0, 8      ' speed, mode, bits
SPI WRITE 4, &H2A, 0, 0, 0   ' a list, an array a(), or a string
SPI READ 3, r$               ' into a string or a numeric array
v = SPI(&H55)                ' write one unit, read one back
PRINT MM.SPISPEED            ' the clock it ACTUALLY got
SPI CLOSE
```

The order of the three pins does not matter. The RP2350 decides which signal each one carries: the controller is `(pin AND 8) = 0` for SPI0, and the role is `pin AND 3` — 0 MISO, 1 CS, 2 SCLK, 3 MOSI. So GP2/GP3/GP4 are SCLK/MOSI/MISO however you write them. On the header SPI0 is **GP0–GP7 and GP34–GP39**; the rest (GP26, GP40, GP42–GP46) are SPI1, which is **the SD card** and is not offered — a program that could take it could take the filesystem out from under itself. This is MMBasic's behaviour too: it asks each pin what it can be rather than fixing an order.

Chip select is yours, exactly as on a PicoMite. A display needs CS held across a whole command-and-data sequence rather than per transfer, so only the program knows when to move it — and a pin is a register write now, not a system call.

`MM.SPISPEED` is worth using. The divisor is `clk_peri / (CPSDVSR × (1 + SCR))` with `CPSDVSR` even, so a request nearly always lands on a neighbouring value: asking for 50 MHz gives 46.875, and anything above `clk_peri / 2` quietly becomes `clk_peri / 2`. The usable steps here are 62.5, 46.875, 37.5, 31.25 MHz and down.

A whole transfer is **one system call whatever its length** — the controller reads your buffer where it lies rather than copying it through the kernel, which is sound here because there is no MMU and the kernel cannot be preempted. A 240×320 screen of 16-bit pixels is 153,600 bytes and goes out in 26 ms at 62.5 MHz.

One limit to know: a **string holds at most 255 bytes**, so a 240-pixel row of RGB565 (480 bytes) takes two writes. The kernel itself has no such limit.

13.7 Not covered

One-wire, `PORT`, and the interrupt statements — along with the editor, `RUN`, `LIST`, `EDIT` and the rest of the immediate-mode environment. The hardware statements are the subject of current work; the immediate-mode ones will never apply, since a translated program is compiled and run rather than typed at a prompt. (`mmedit` provides the editing they existed for.)

Of the graphics, `MODE`, `COLOUR`, `PIXEL` (including the array form), `LINE`, `CIRCLE`, `BOX`, `RBOX`, `TRIANGLE` (its drawing form — `SAVE`/`RESTORE` need the interpreter's blit buffers), `ARC`, `RGB()`, `FRAMEBUFFER`, `PRINT @`, `TEXT`, `FONT`, `CLS [colour]` and `MAP` (statement and function) are done; `BLIT` is not yet, nor are `FRAMEBUFFER LAYER` and `FRAMEBUFFER MERGE`. `TEXT` draws in any of MMBasic's nine built-in fonts but only in its normal and vertical orientations — the three that rotate the character itself are accepted and drawn normally.

Of the pins, `SETPIN n, DIN|DOUT|AIN|ARAW|INTH|INTL|INTB|PWM|OFF`, `PIN(n) =` and `PIN(n)` are done, and `PWM slice, freq, duty [, duty2]` with `PWM slice, OFF`, and `SERVO slice, position [, position2]` with `SERVO slice, OFF`; the frequency and counting modes of `SETPIN`, and `PWM SYNC`, are not. Interrupt handlers must be SUBs — MMBasic's label and line-number targets are refused, and with them `IRETURN`, which a SUB handler never needs.

Of the interrupts, `SETPIN INTH|INTL|INTB`, `SETTICK` (all four timers, `PAUSE`, `RESUME`, off) and `ON KEY` (both forms) are done. They are MMBasic's poll, checked between statements, with its priority order and its no-nesting rule. Three divergences, all named where they are described: `SETTICK` fires at the period asked for rather than MMBasic's period-plus-a-millisecond; the console is checked at most every 5 ms; and keys reach a program only when a line is complete, which is the tty driver's doing and limits `ON KEY` to command-at-a-time input. `ON PS2`, `INTERRUPT`, `PID` and `SENSORFUSION` have nothing to notify on this machine. The analogue pair need an ADC pin (GP40–GP46 here), and `AIN` uses MMBasic's own ten-sample sort-and-discard filter. `PIN()` returns a float in every mode rather than MMBasic's integer for digital and `ARAW`, and pins are *owned*: `SETPIN` claims from the kernel, only the I/O header may be claimed, and the pin is released and reset when the program ends however it ends. `OPTION VCC` is not supported, so `AIN` always scales by 3.3 V.

Of the sound, `PLAY MP3`, `PLAY VOLUME` and `PLAY STOP` are done — `PLAY TONE`, `WAV`, `FLAC`, `MOD`, `MIDI`, `SAMPLE` and `EFFECT` are not, and neither is the four-channel `SOUND` synthesiser the kernel provides to BBC BASIC. `PLAY VOLUME` takes one level rather than one per channel.

In a graphics mode a program's `PRINT` now draws the characters into whatever is being drawn on, as MMBasic does, so text goes into the framebuffer with everything else. What is still missing is `OPTION CONSOLE`, to say whether a program's output should go to the screen, the serial port, or both.